Advanced Regression Model Comparison & Optimization Platform

Overview

The objective of this project is to develop machine learning application that predicts house prices using multiple regression algorithms. The project exploring relationships between variables, and building multiple regression models for comparison.

Dataset : **House Prices – Advanced Regression Techniques dataset**

target variable : **SalePrice**.

csv used : **train.csv , test.csv**

Throughout the project I Use different regression models to train the models and using the same processed dataset.

Data Understanding (Phase 1)

Objective

The first phase focused on understanding the dataset and identifying the characteristics of each feature before performing any preprocessing or model building.

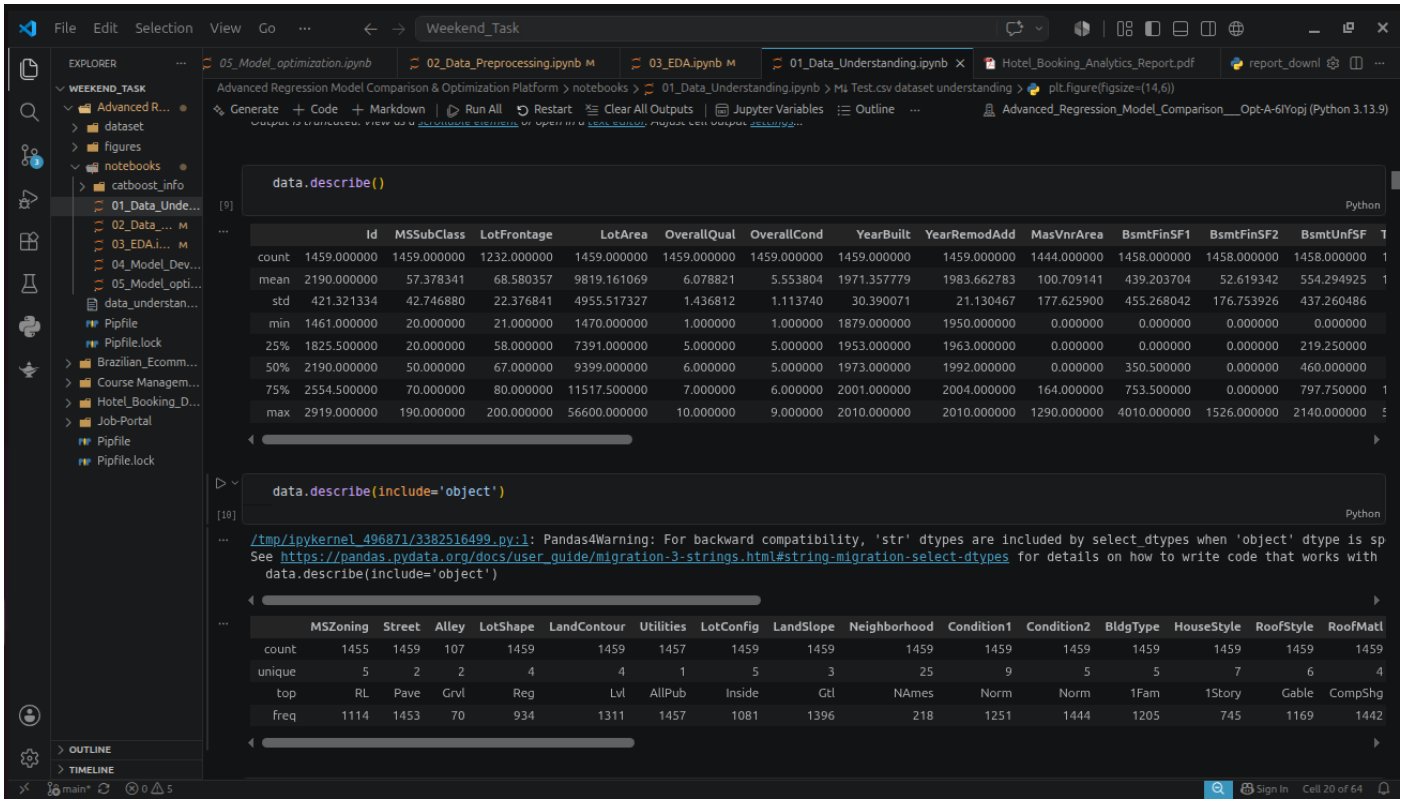
Tasks Performed

- Loaded the training and test datasets.
- Explored the business problem and understood the prediction objective.
- Studied the dataset structure, feature names, and data types.
- Classified features into numerical and categorical variables.
- Identified missing values across all columns.
- Checked for duplicate records.
- Analyzed the distribution of the target variable (SalePrice).

Visualizations

The following visualizations were created during this phase:

- Histograms
- Correlation Heatmap
- Feature Correlation Graph
- Missing Value Analysis



```
data.describe()
```

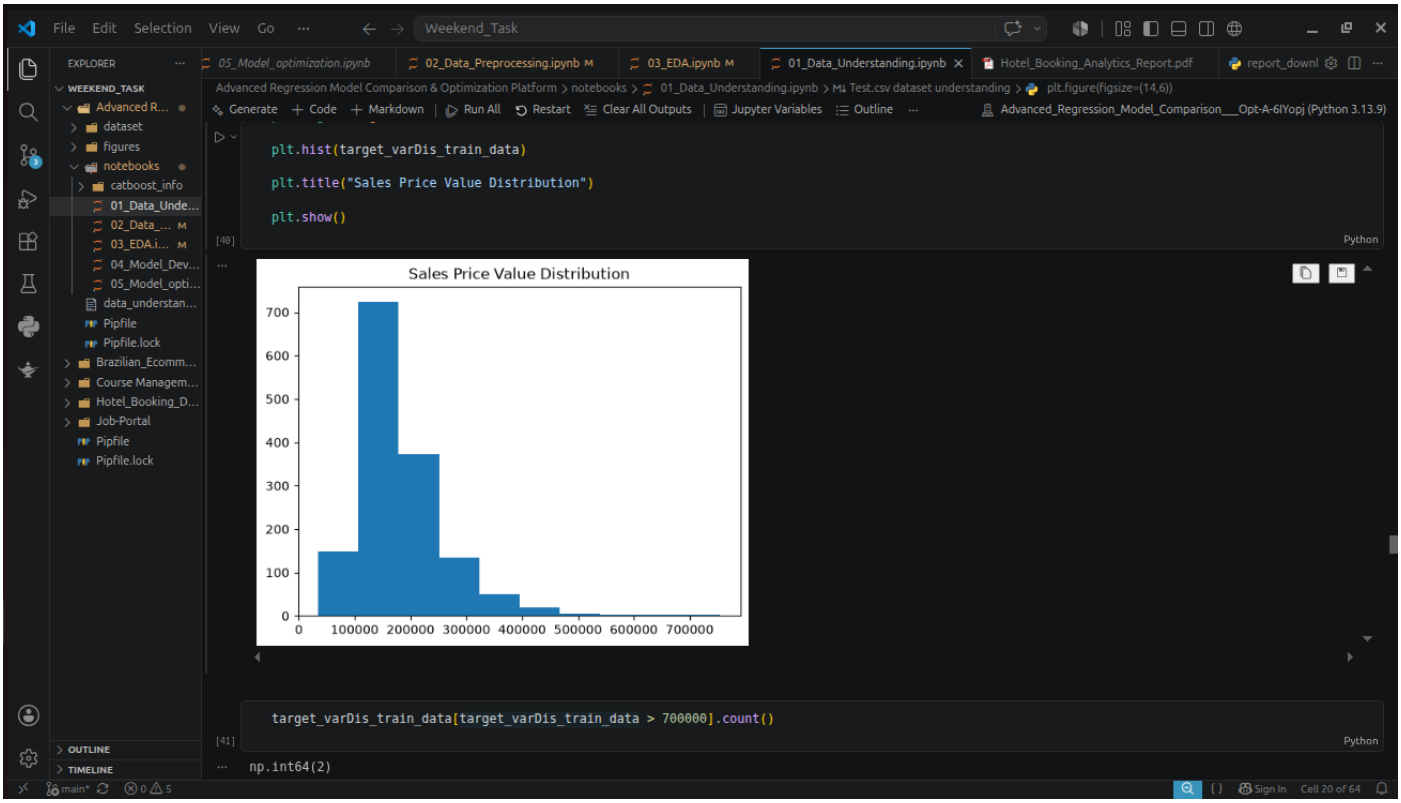
	Id	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	YearRemodAdd	MasVnrArea	BsmtFinSF1	BsmtFinSF2	BsmtUnfSF
count	1459.000000	1459.000000	1232.000000	1459.000000	1459.000000	1459.000000	1459.000000	1459.000000	1444.000000	1458.000000	1458.000000	1458.000000
mean	2190.000000	57.378341	68.580357	9819.161069	6.078821	5.553804	1971.357779	1983.662783	100.709141	439.203704	52.619342	554.294925
std	421.321334	42.746880	22.376841	4955.517327	1.436812	1.113740	30.390071	21.130467	177.625900	455.268042	176.753926	437.260486
min	1461.000000	20.000000	21.000000	1470.000000	1.000000	1.000000	1879.000000	1950.000000	0.000000	0.000000	0.000000	0.000000
25%	1825.500000	20.000000	58.000000	7391.000000	5.000000	5.000000	1953.000000	1963.000000	0.000000	0.000000	0.000000	219.250000
50%	2190.000000	50.000000	67.000000	9399.000000	6.000000	5.000000	1973.000000	1992.000000	0.000000	350.500000	0.000000	460.000000
75%	2554.500000	70.000000	80.000000	11517.500000	7.000000	6.000000	2001.000000	2004.000000	164.000000	753.500000	0.000000	797.750000
max	2919.000000	190.000000	200.000000	56600.000000	10.000000	9.000000	2010.000000	2010.000000	1290.000000	4010.000000	1526.000000	2140.000000

```
data.describe(include='object')
```

```
/tmp/ipykernel_496871/3382516499.py:1: Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is sp
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with
  data.describe(include='object')
```

	MSZoning	Street	Alley	LotShape	LandContour	Utilities	LotConfig	LandSlope	Neighborhood	Condition1	Condition2	BldgType	HouseStyle	RoofStyle	RoofMatl
count	1455	1459	107	1459	1459	1457	1459	1459	1459	1459	1459	1459	1459	1459	1459
unique	5	2	2	4	4	1	5	3	25	9	5	5	7	6	4
top	RL	Pave	Grvl	Reg	Lvl	AllPub	Inside	Gtl	NAmes	Norm	Norm	1Fam	1Story	Gable	CompShg
freq	1114	1453	70	934	1311	1457	1081	1396	218	1251	1444	1205	745	1169	1442

data understanding



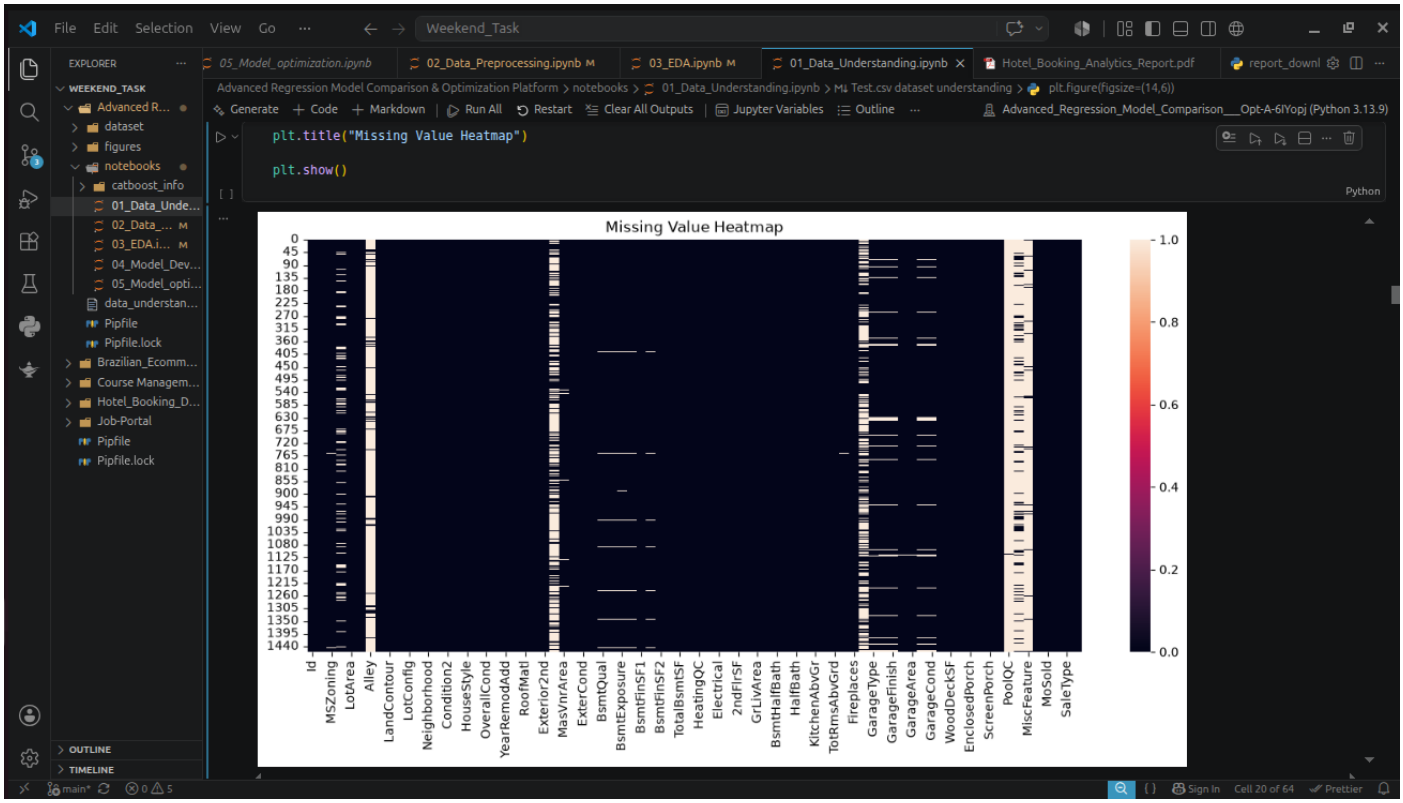
```
plt.hist(target_varDis_train_data)

plt.title("Sales Price Value Distribution")

plt.show()
```



```
target_varDis_train_data[target_varDis_train_data > 700000].count()
```

```
np.int64(2)
```

Visualization of SalesPrice Distribution



Missing Value HeatMap

Data Preprocessing (Phase 2)

Objective

The objective of this phase was to prepare the dataset for machine learning by handling missing values, reducing the effect of outliers, encoding categorical variables, scaling numerical features, and selecting relevant features.

csv used : **train.csv , test.csv**

Important Notes:

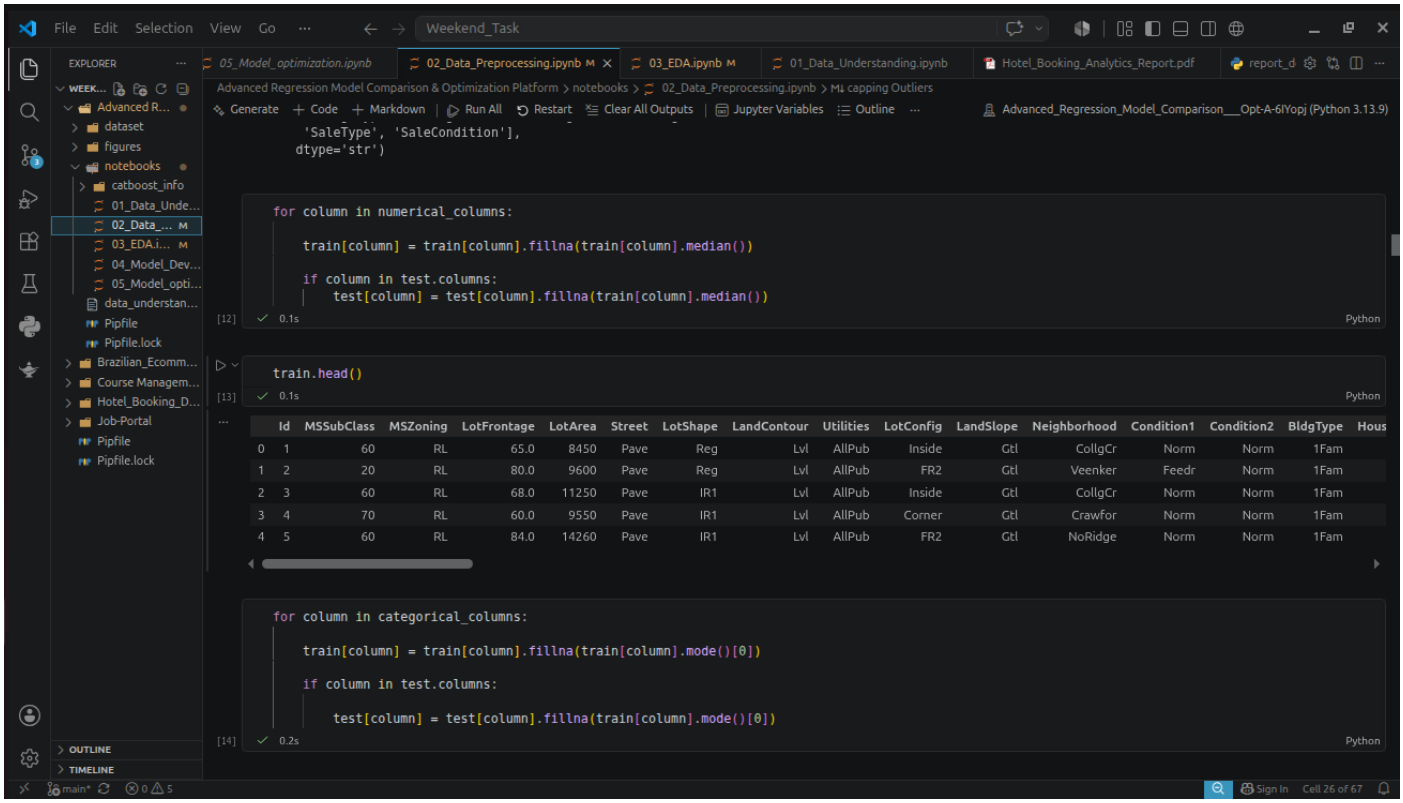
What are the preprocessing techniques done for the train dataset is same as done for the test dataset. Only the Target Variable **SalesPrice** Columns is Differ.

Tasks Performed

Handling Missing Values

Missing values were treated according to the nature of each feature.

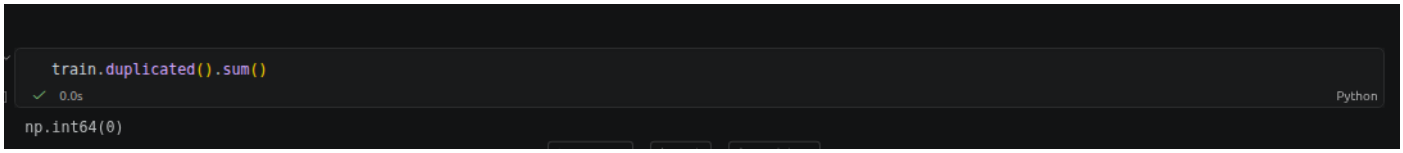
- Numerical features were filled using the median.
- Categorical features were filled using the most frequent category.



Handle Missing values with Median and Mode

Removing Duplicate Records

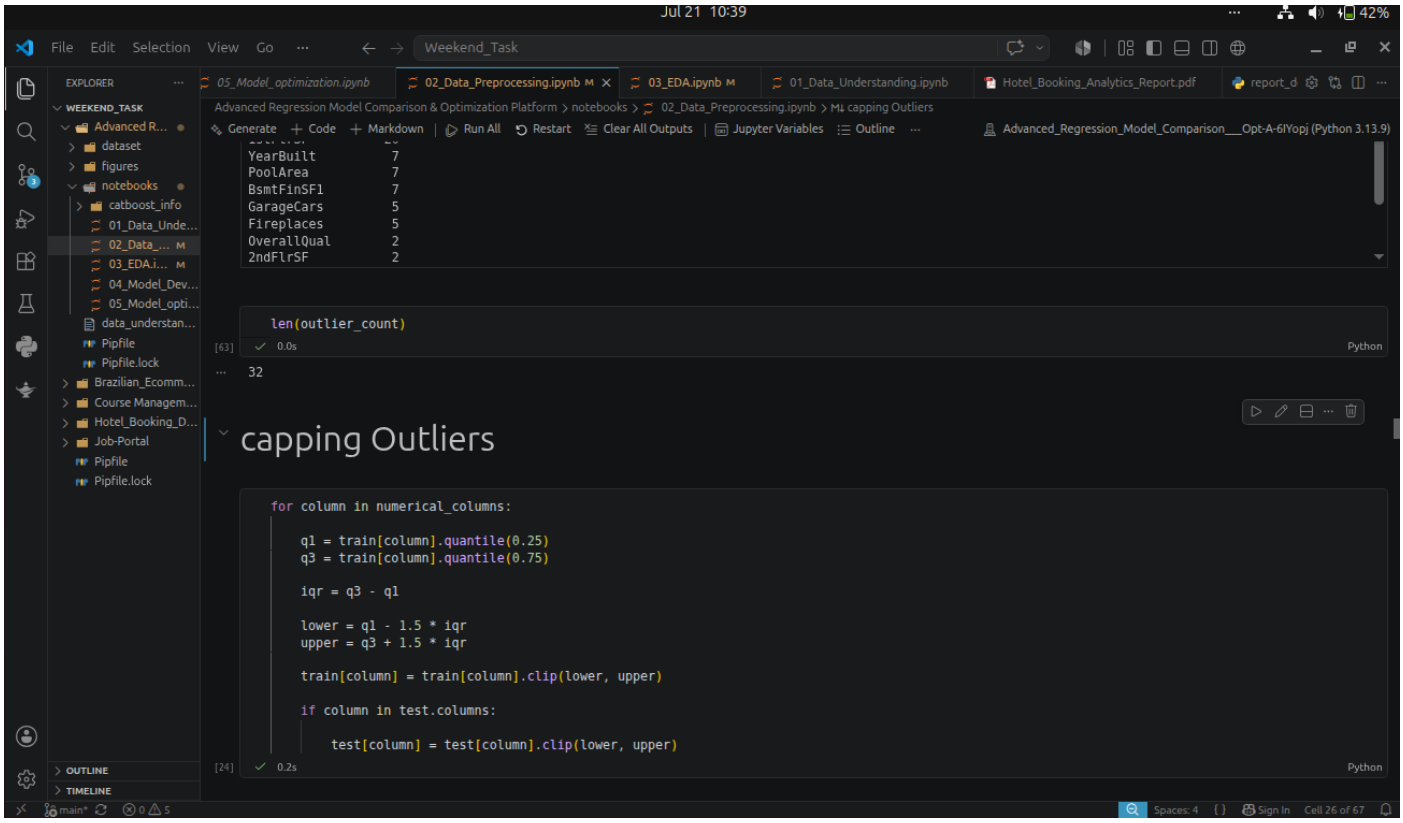
The dataset was checked for duplicate observations to ensure data quality. No duplicate rows found in the both Train and Test dataset.



0 duplicate values

Outlier Treatment

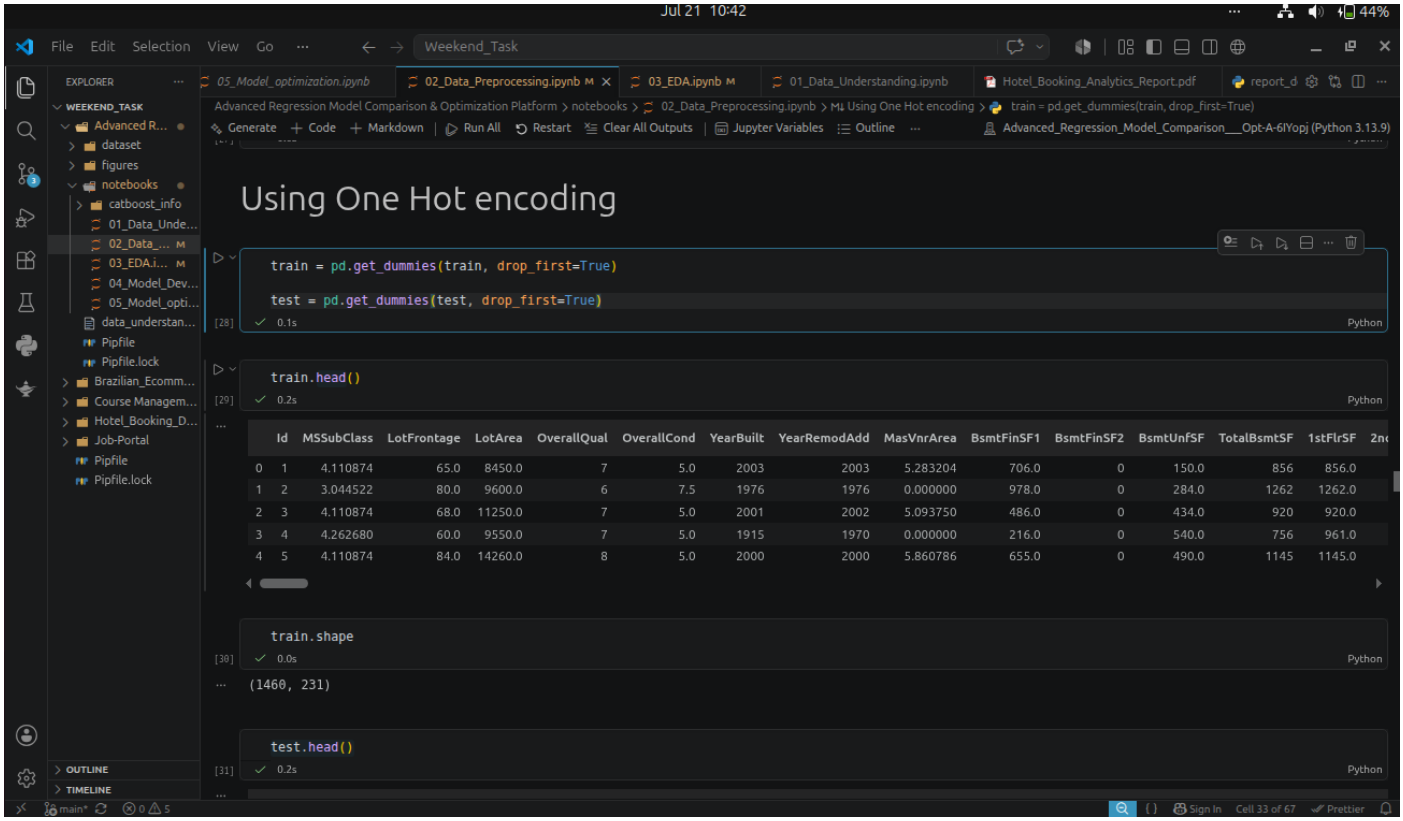
- Outliers in numerical features were handled using the **Interquartile Range (IQR)** method.
- Instead of removing observations, extreme values were capped using the lower and upper IQR limits. Totally 32 columns have outliers.



Outlier handle by capping with lower and higher

Encoding Categorical Variables

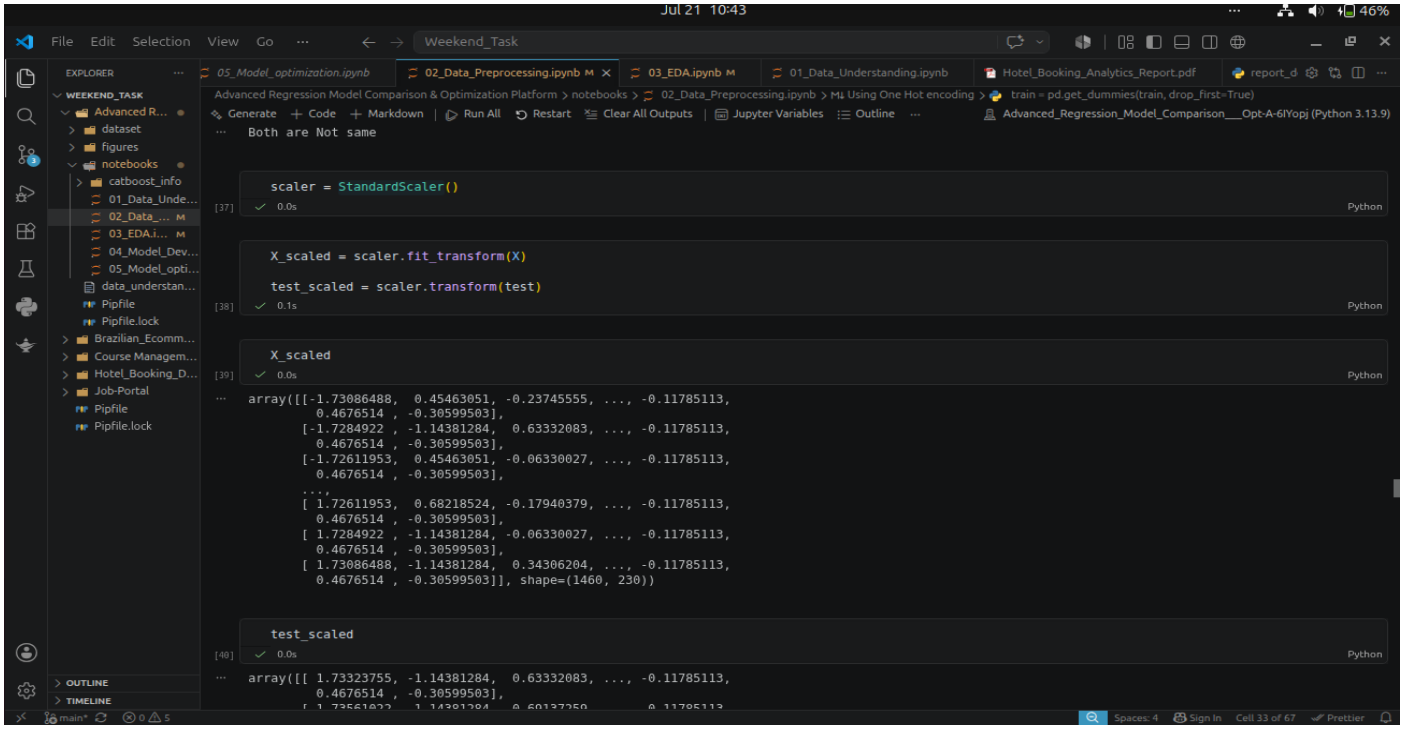
- Categorical variables were converted into numerical form using **One-Hot Encoding**.
- This allowed machine learning algorithms to process categorical information without introducing an artificial order among categories.



One Hot Encoding

Scaling Numerical Features

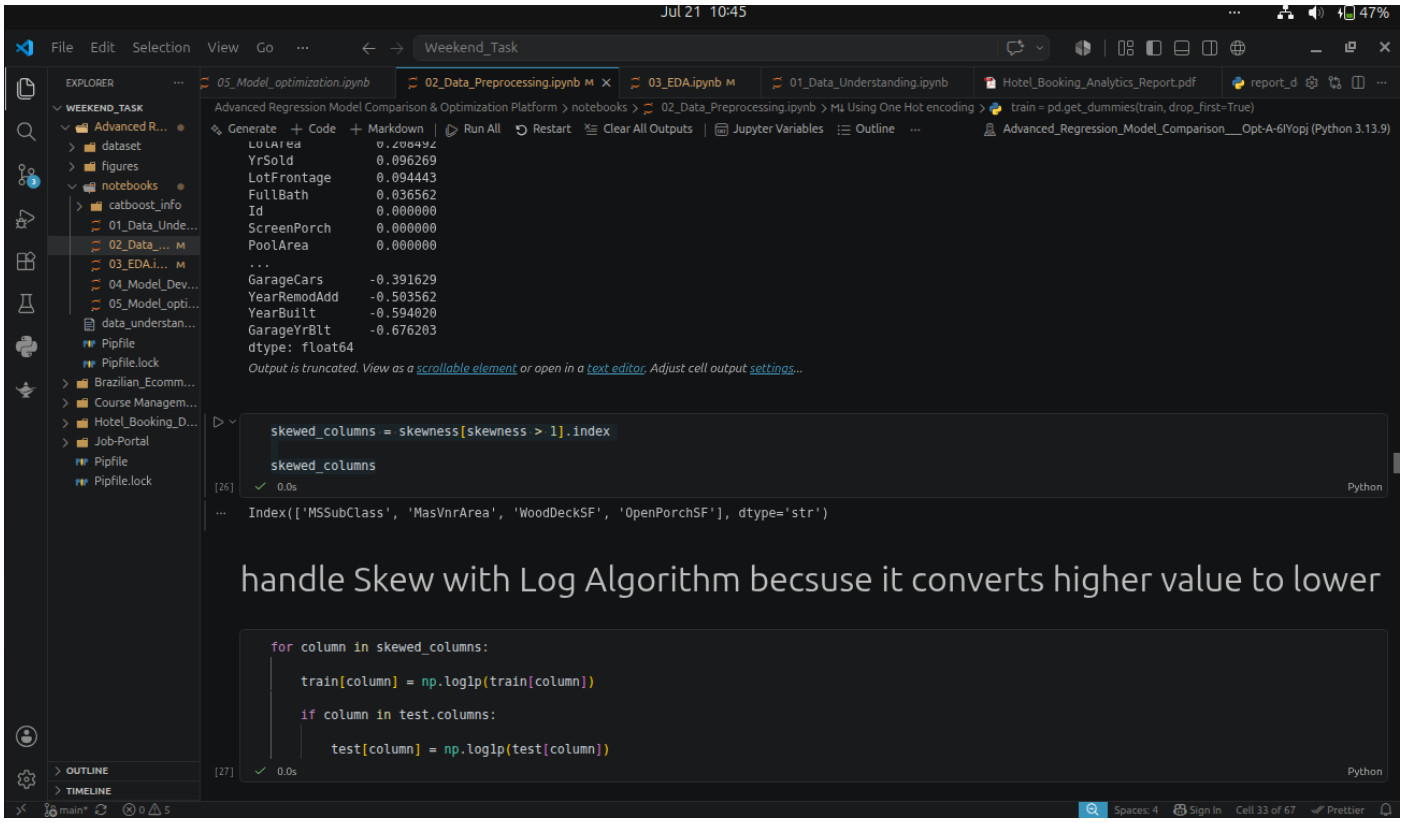
- Numerical features were standardized using **StandardScaler**.
- Scaling ensured that variables measured on different ranges contributed equally during model training.



standard scale

Handling Skewed Distributions

Highly skewed numerical features were transformed using logarithmic transformation (log1p) to make their distributions more symmetric.



Handle Skewness

Feature Selection

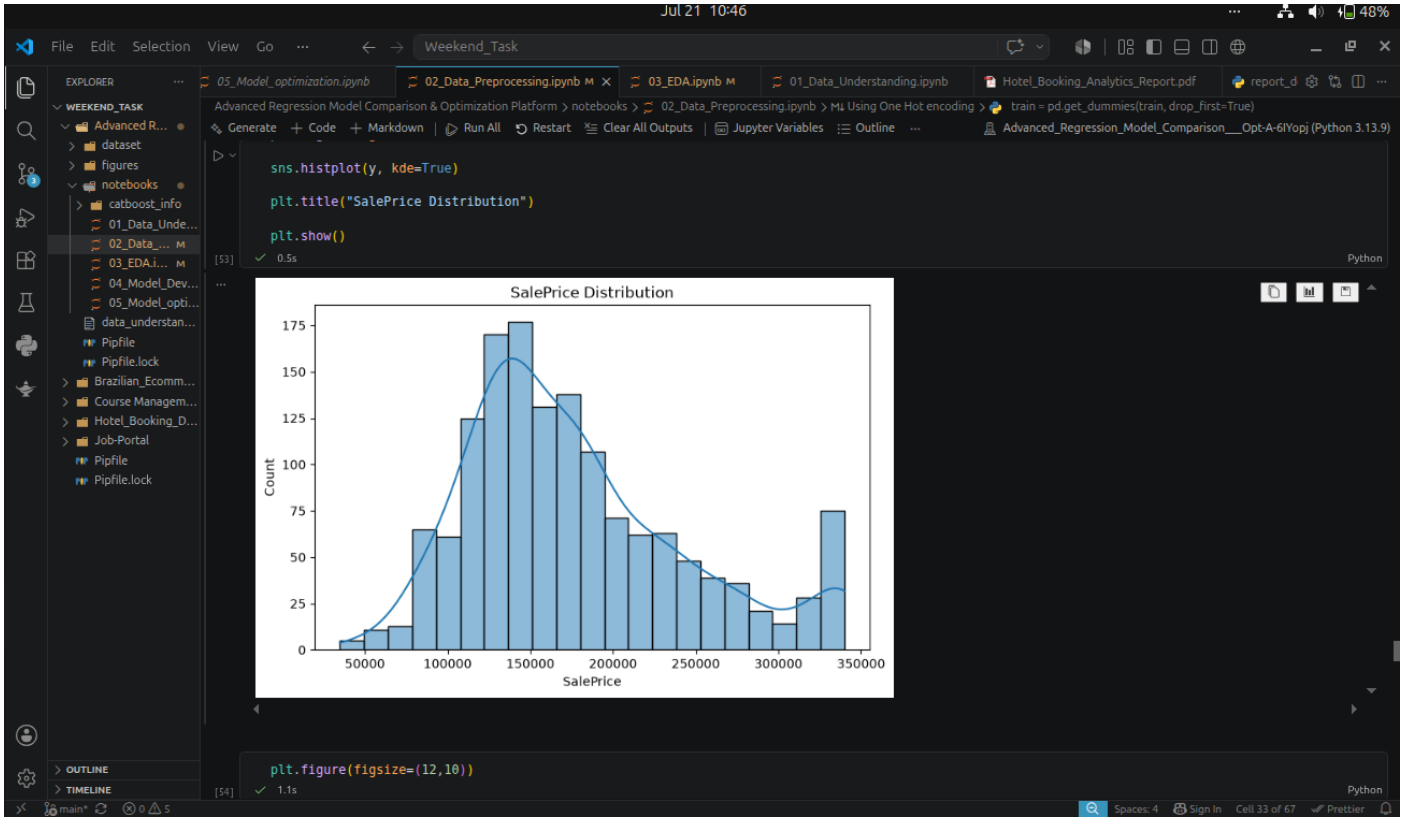
Correlation analysis was used to identify the features most strongly related to house prices. Less informative features were removed to simplify the dataset while retaining predictive information.

Note : After the Preprocessing the new processed dataset is stored in the name train_processed.csv

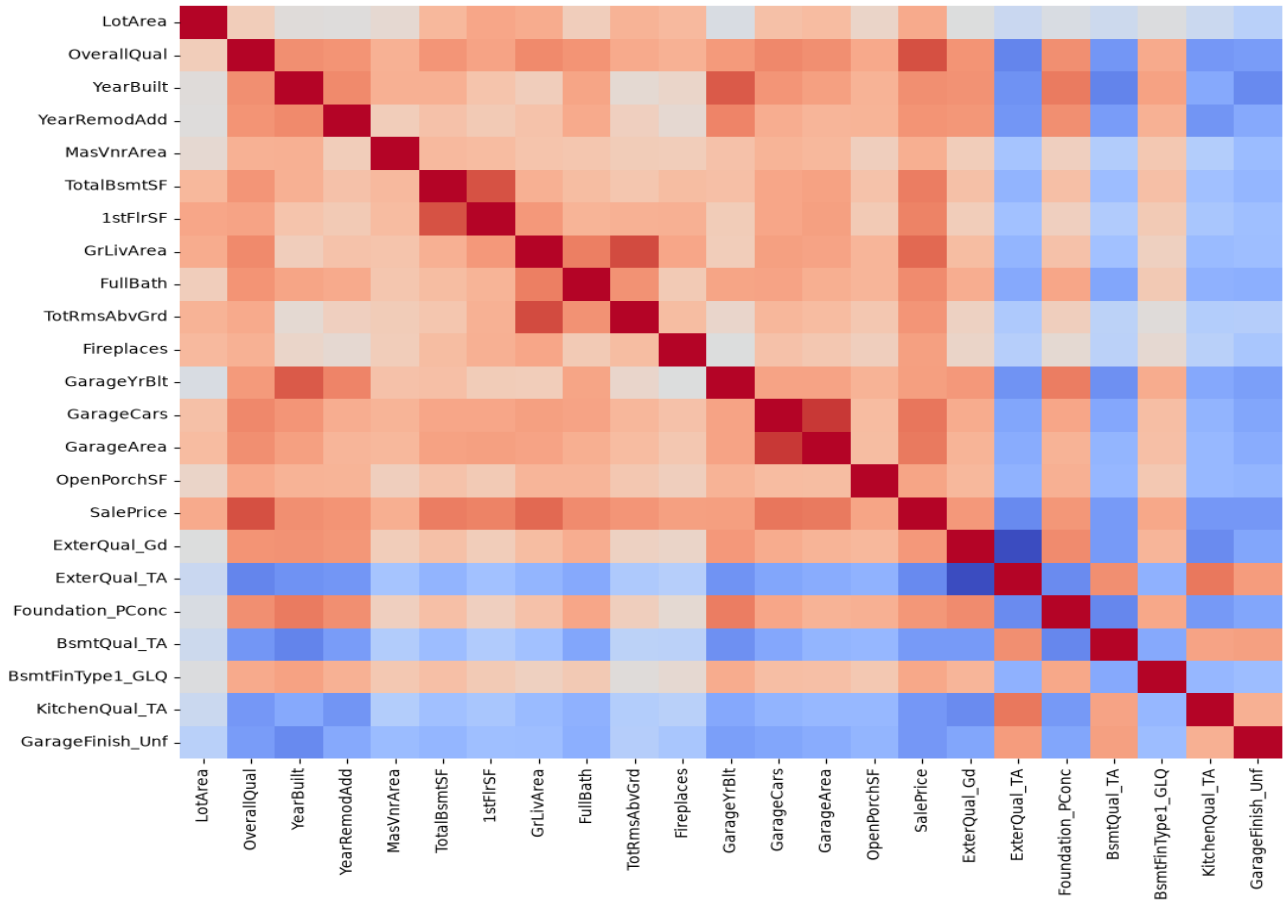
Visualizations

The preprocessing phase included:

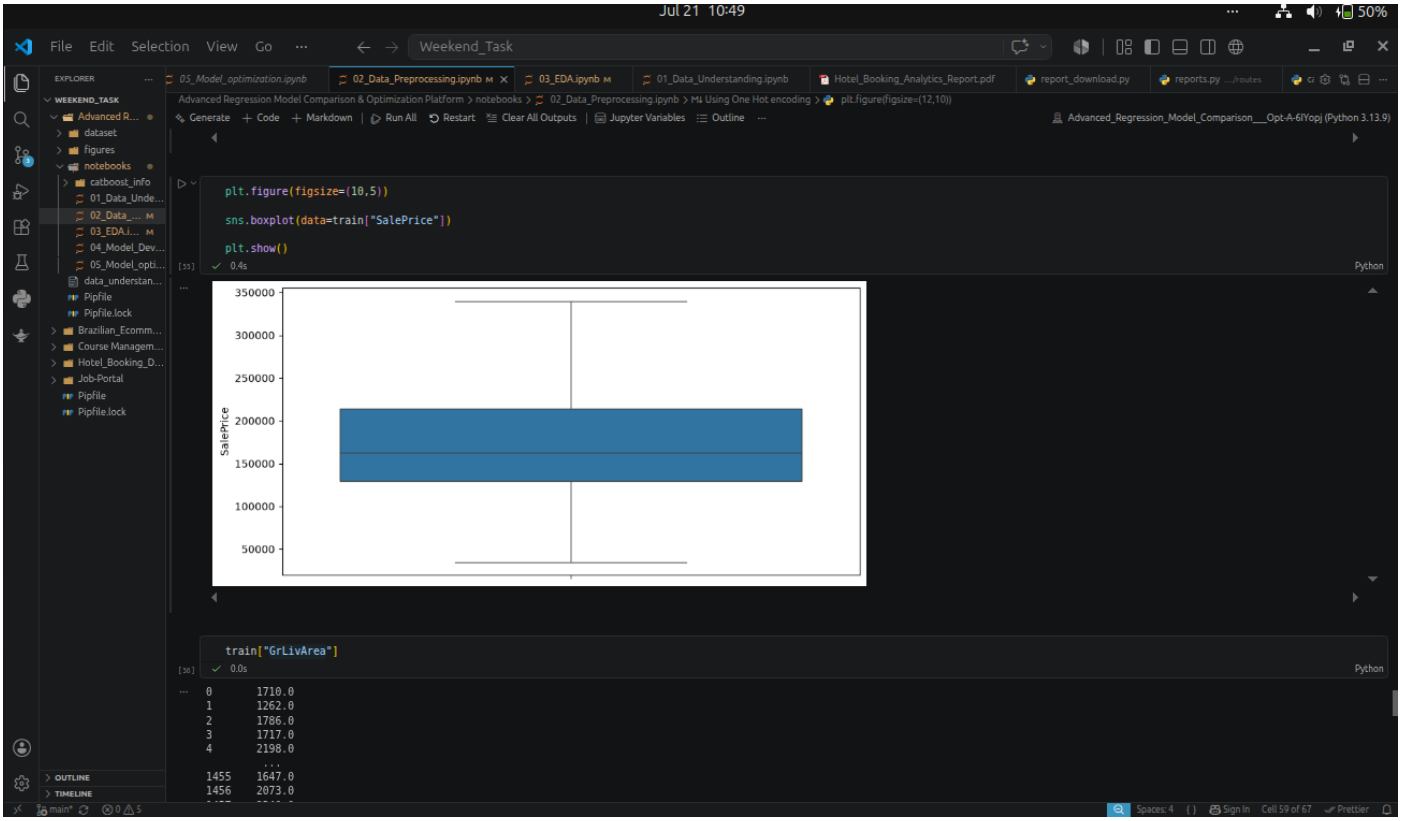
- Distribution comparison before and after transformation
- Correlation matrix
- Box plots after outlier treatment



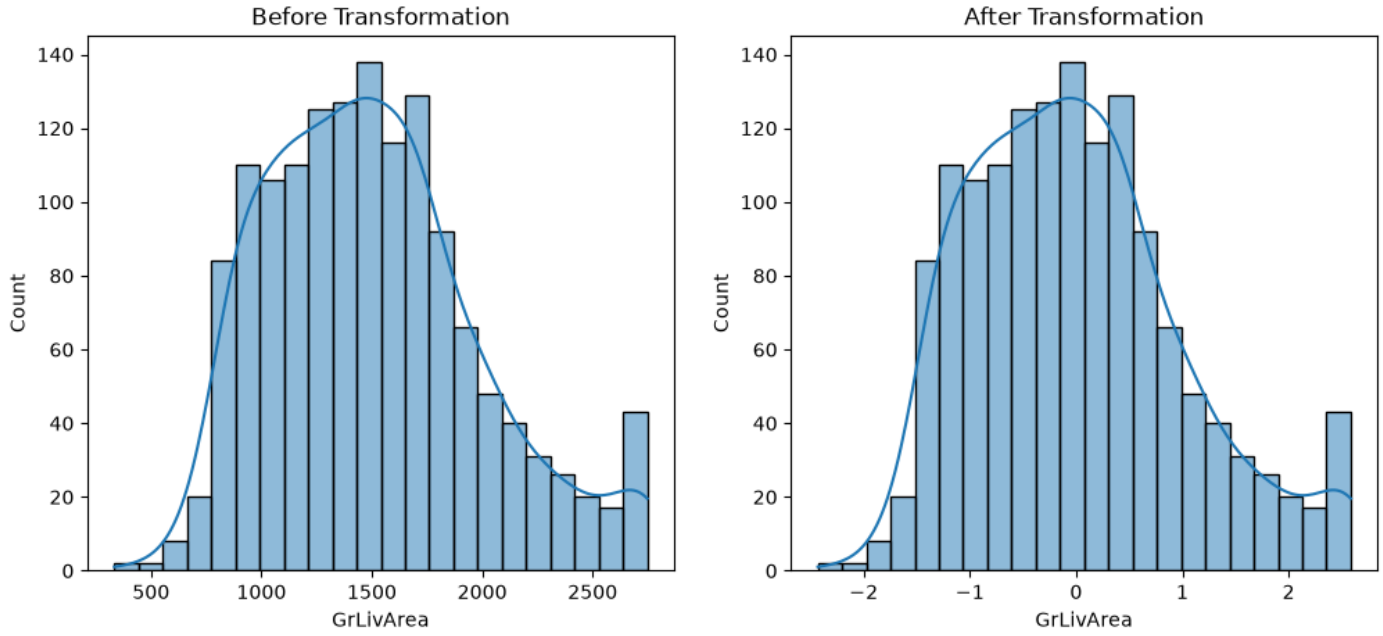
SalesPrice Distribution



Numerical features HeatMap



Box plot for the SalesPrice Outliers



GrLiveArea Before(cleaned) and After Transformation(scaled)

Exploratory Data Analysis (Phase 3)

Objective

The objective of this phase was to study relationships between variables and understand how different features influence house prices.

csv used : train.csv

Tasks Performed

Feature vs Target Analysis

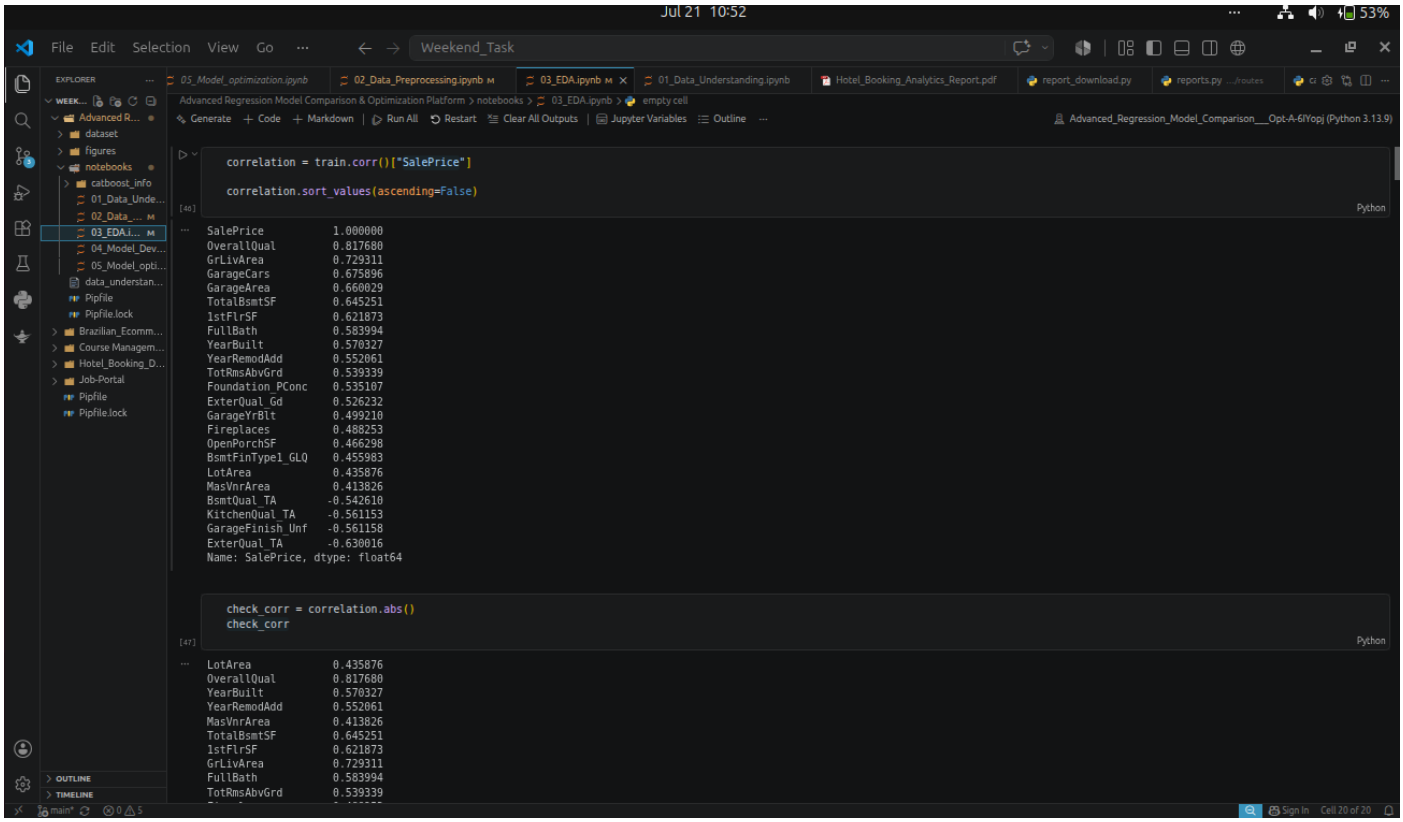
Important numerical features were compared against the target variable using scatter plots.

This helped identify variables with strong positive or negative relationships with house prices.

Correlation Analysis

A correlation matrix was generated to understand linear relationships among numerical variables.

Highly correlated variables were identified for further investigation.



Correlation check

Important Feature

Correlation values were used as an initial measure of feature importance.

Features with the strongest relationship to the target variable were identified for further modeling.

```
important_features = [
    "GrLivArea",
    "TotalBsmtSF",
    "GarageArea",
    "SalePrice",
    "GarageCars",
    "OverallQual"
]
```

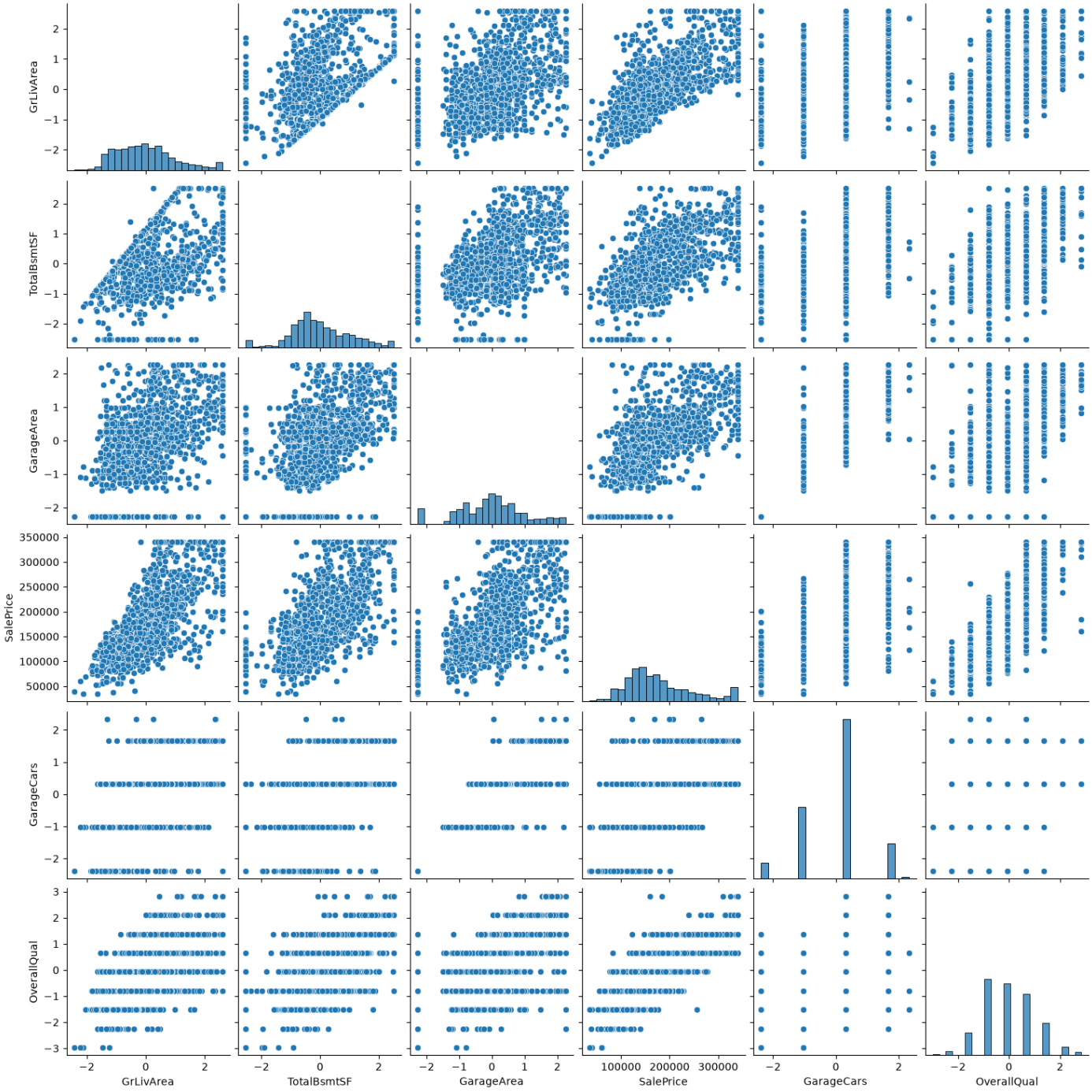
Relationship Between Important Variables

Pair plots and heatmaps were created to visualize interactions among the most influential variables.

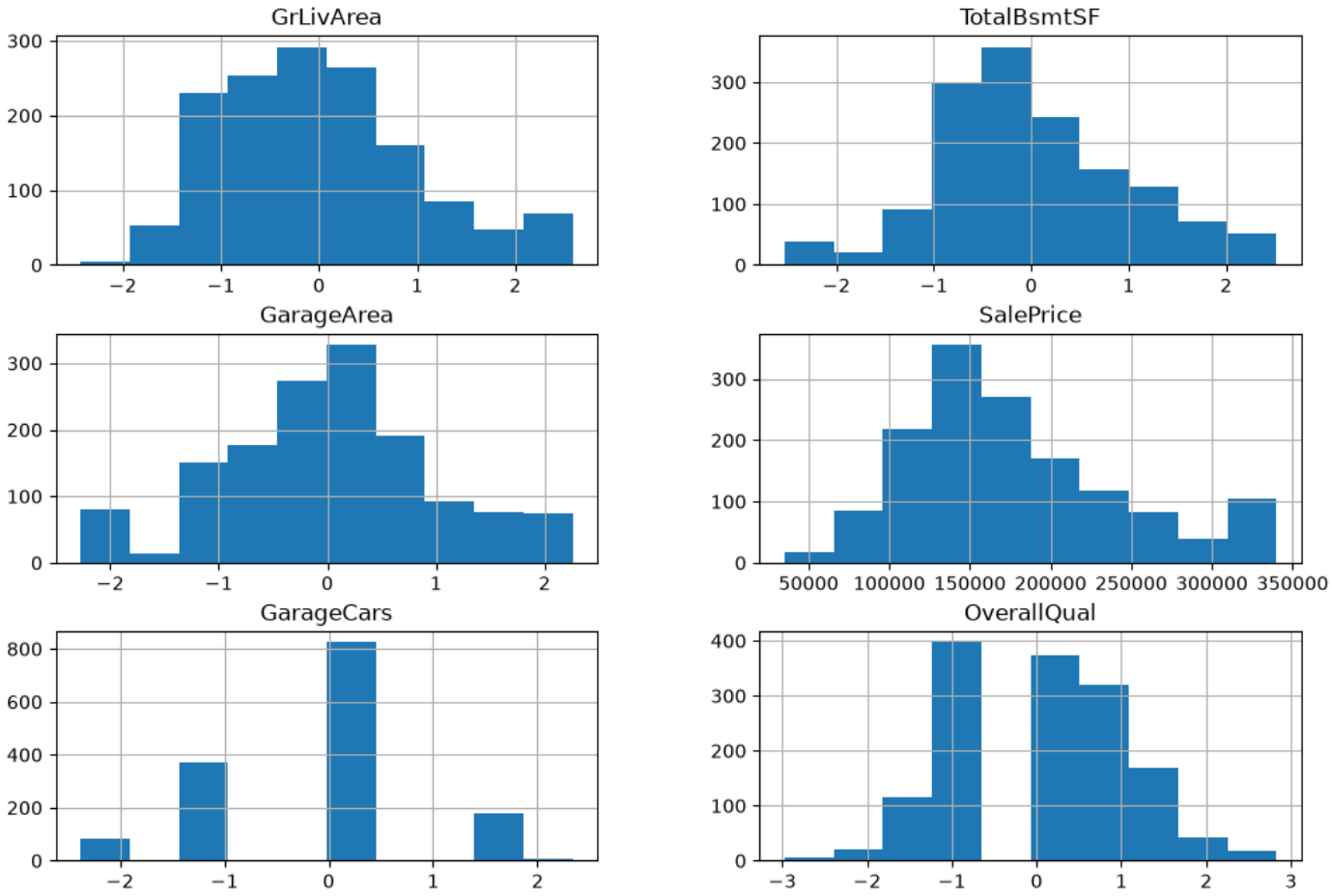
Visualizations

The following visualizations were created:

- Scatter Plots
- Correlation Heatmaps
- Histograms
- Pair Plots
- Distribution Charts



Pair Plots for important Features



Histogram representation of the Important Features

Model Development (Phase 4)

Objective

The objective of this phase was to build multiple regression models using the same processed dataset and establish baseline performance for each algorithm.

Csv used : train_processed.csv

Dataset Preparation

The processed dataset from Phase 2 was loaded.

The target variable (SalePrice) was separated from the input features.

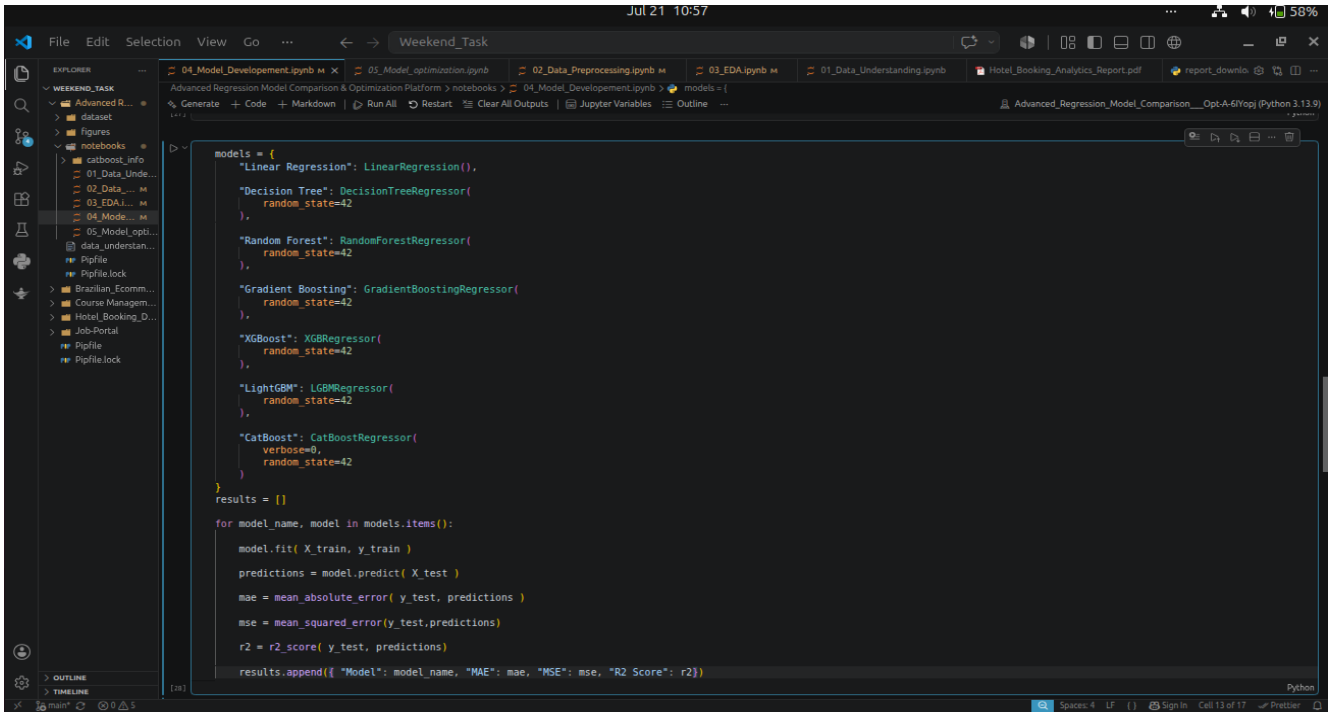
The dataset was divided into training and testing sets using an 80:20 ratio.

Regression Models Implemented

The following regression algorithms were trained:

- Linear Regression
- Decision Tree Regressor
- Random Forest Regressor
- Gradient Boosting Regressor
- XGBoost Regressor
- LightGBM Regressor
- CatBoost Regressor

Each model received the same training and testing data to ensure consistent evaluation.



Model Trained and Error Calculated

Model Evaluation

The performance of each model was evaluated using:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score

Top	Model	MAE	MSE	R2 Score
1	CatBoost	12765.161892	3.294861e+08	0.932610
2	LightGBM	13997.545370	3.822302e+08	0.921822
3	Gradient Boosting	14113.775135	3.922355e+08	0.919776
4	Random Forest	14800.120291	4.251961e+08	0.913034
5	Linear Regression	15707.820714	4.661069e+08	0.904667
6	XGBoost	15772.649827	5.356677e+08	0.890440
7	Decision Tree	22591.566781	1.049320e+09	0.785382

MODEL IMPROVEMENT & OPTIMIZATION(Phase 5)

Introduction

The baseline regression models developed in the previous phase provided good prediction accuracy. However, further improvements were required to enhance model performance, reduce redundancy in the dataset, and improve the models' ability to generalize to unseen data.

To achieve this, several optimization techniques were applied, including feature engineering, feature selection, data transformation, outlier treatment, missing value handling, multicollinearity analysis, cross validation, and hyperparameter tuning. These techniques helped improve the overall quality of the dataset and produced more reliable regression models.

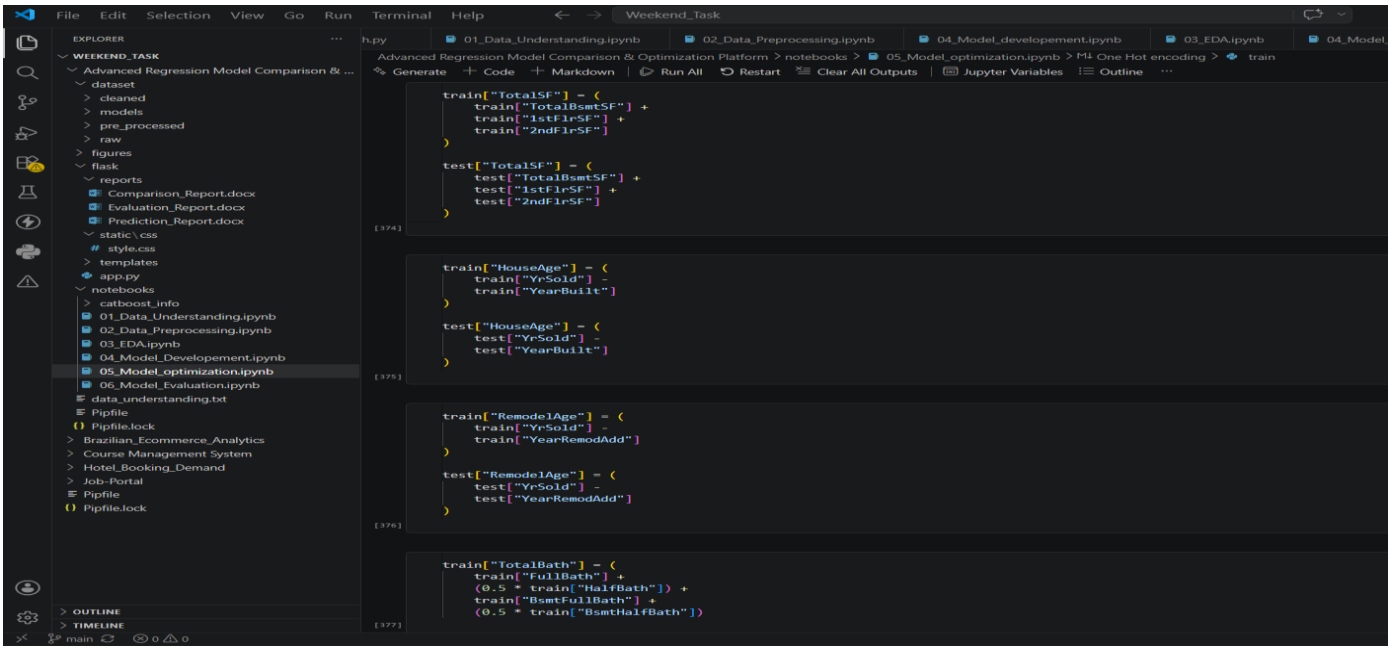
Feature Engineering

Feature engineering was performed to create new features from the existing dataset. These newly created features provided additional information that helped the regression models better understand the relationship between house characteristics and selling price.

Some important engineered features included:

- TotalSF
- HouseAge
- RemodelAge
- TotalBath
- TotalPorchSF

These features were created by combining related attributes and were later used during model training.

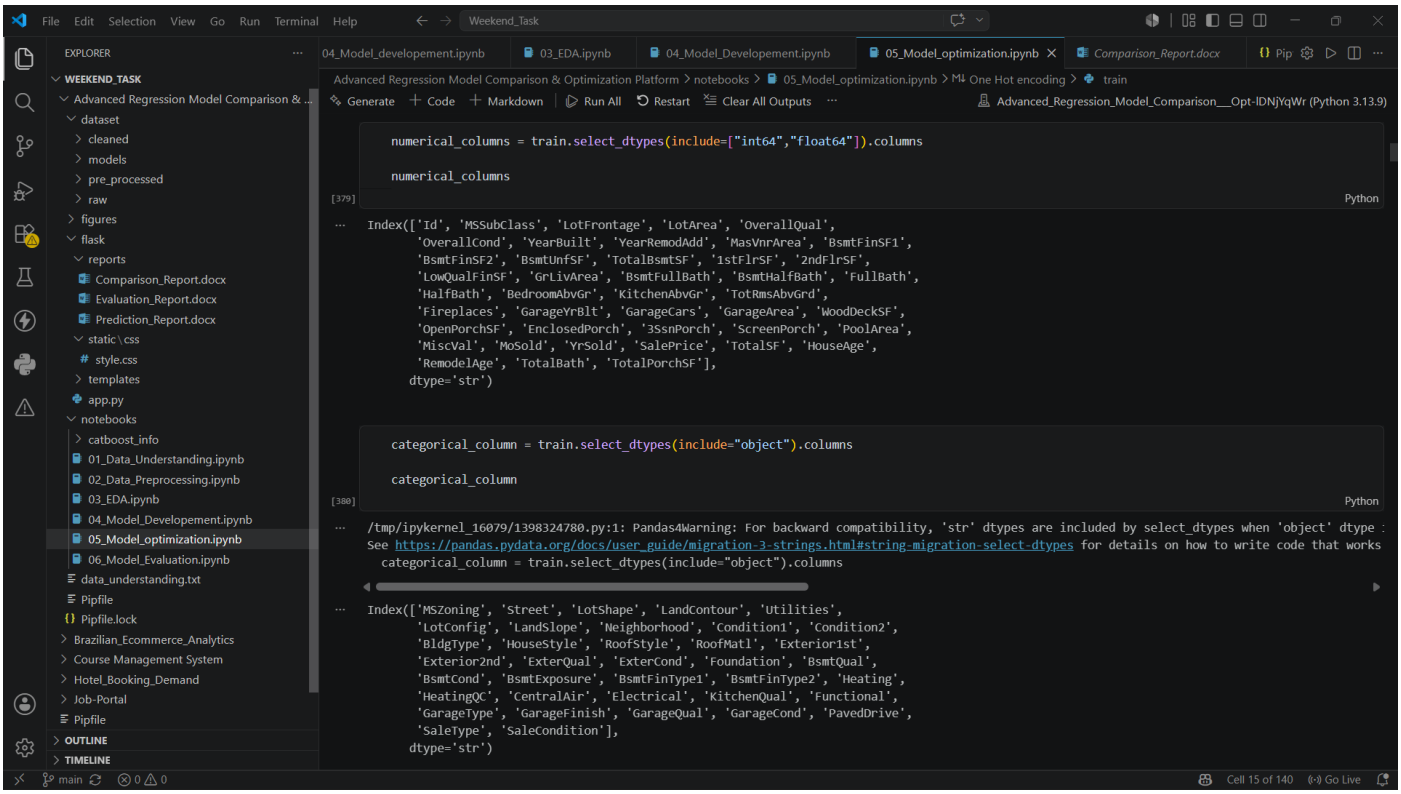


Feature Selection

After feature engineering, feature selection was performed to identify the most influential variables affecting house prices.

Feature importance scores obtained from the trained model were used to rank the features. The top-ranked features contributed the most to prediction accuracy and were selected for the final deployed model.

The final prediction system was built using only the most important features, reducing model complexity while maintaining good performance.

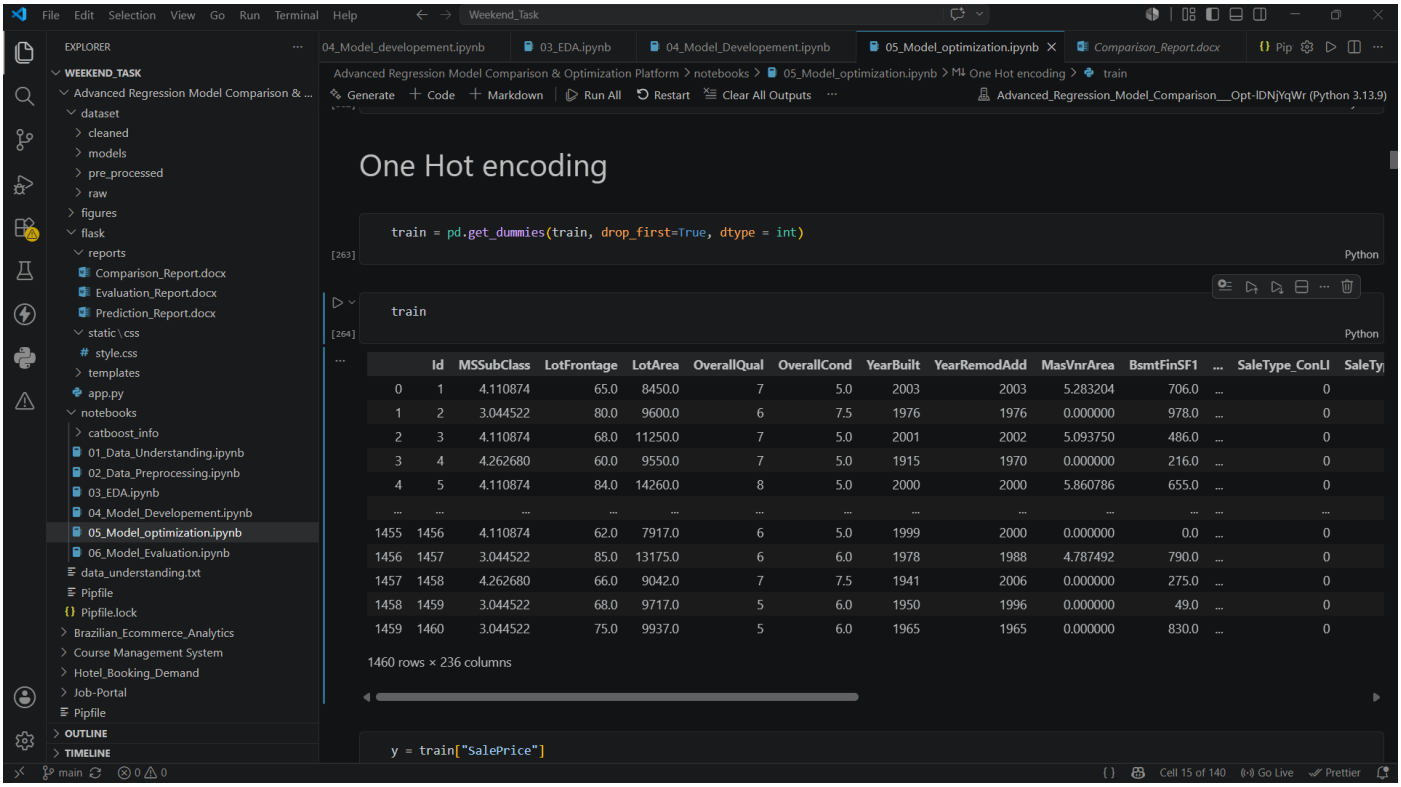


Feature Selection

Data Transformation

Data transformation was applied to improve the distribution of numerical features. Highly skewed variables were transformed using logarithmic transformation, resulting in a more balanced distribution.

This preprocessing step helped improve the learning capability of regression algorithms.



Data Transformation

Outlier Treatment

Outliers were identified using the Interquartile Range (IQR) method.

Instead of removing observations, extreme values were capped within the acceptable range. This approach reduced the influence of abnormal values while preserving the overall dataset.

Missing Value Handling

After feature engineering and preprocessing, the dataset was checked for missing values.

Numerical features were filled using the median, while categorical features were filled using the most frequent category. This ensured that the dataset contained no missing values before model training.

Multicollinearity Analysis

Multicollinearity was analyzed using the Variance Inflation Factor (VIF).

Features with very high VIF values indicated strong correlations with other variables and were removed to reduce redundancy in the dataset. Features with undefined (NaN) VIF values were

retained, as they represented constant or low-variance features that did not affect the tree-based regression models.

	Feature	VIF
0	Id	1.173312
1	MSSubClass	15.786954
2	LotFrontage	2.668480
3	LotArea	3.636655
4	OverallQual	5.243093
...	...	...
230	SaleCondition_AdjLand	1.472638
231	SaleCondition_Alloca	1.375566
232	SaleCondition_Family	1.140551
233	SaleCondition_Normal	0.005793
234	SaleCondition_Partial	46.790101

235 rows × 2 columns

VIF value DF

Cross Validation

Five-fold Cross Validation was performed to evaluate the stability and generalization capability of each regression model.

The average Cross Validation score provided a more reliable estimate of model performance than a single train-test split and helped identify models that generalized well to unseen data.

	Model	Average CV Score
0	Gradient Boosting	0.842758
1	LightGBM	0.839737
2	CatBoost	0.839737
3	Random Forest	0.830443
4	XGBoost	0.829643
5	Linear Regression	0.826077
6	Decision Tree	0.649256

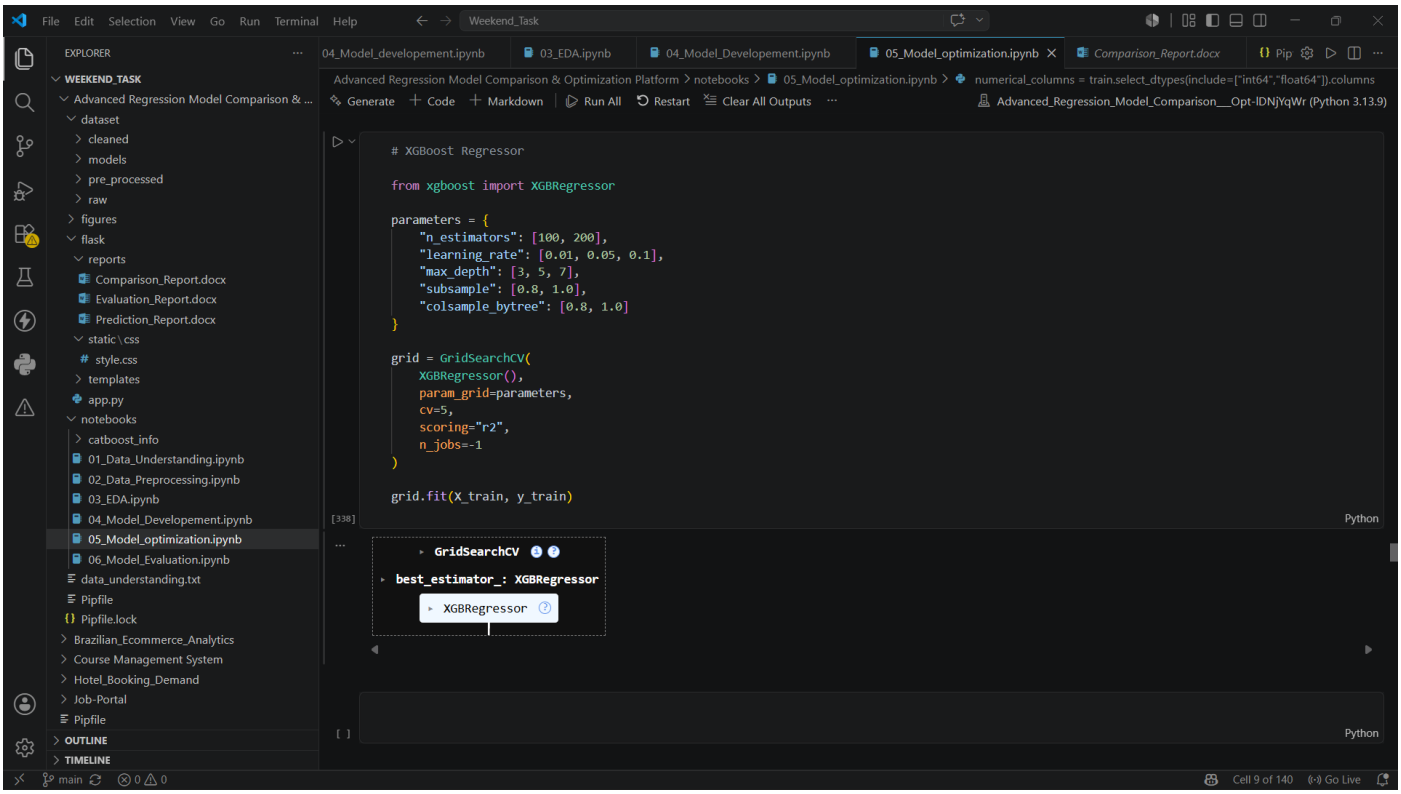
Cross Validation Table

Hyperparameter Tuning

Hyperparameter tuning was performed using **GridSearchCV** and **RandomizedSearchCV** to identify the optimal parameter combinations for each regression model.

After tuning, the models were retrained and evaluated using the optimized parameters. Although the optimized models showed a slight reduction in R^2 score compared to the baseline models, they

demonstrated improved generalization and were trained using a refined feature set created through feature engineering and feature selection.



Hyper parameter Tuning

MODEL EVALUATION, COMPARISON & INTERPRETATION(Phase 6 and 7)

Introduction

After optimizing the regression models, the next step was to evaluate their performance using the test dataset. The models were compared using standard regression evaluation metrics to identify the most suitable model for house price prediction.

In addition to performance evaluation, the models were interpreted by analyzing feature importance, comparing prediction accuracy, and selecting the final model for deployment in the Flask web application.

Model Evaluation

Each regression model was evaluated using the same testing dataset to ensure a fair comparison. The following performance metrics were used:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- R^2 Score
- Cross Validation Score

These metrics helped measure the prediction accuracy, error rate, and generalization capability of each model.

Model	MAE	MSE	RMSE	R2 Score	CV Score
XGBoost	16603.658671	5.201392e+08	22806.559507	0.893616	0.829643
LightGBM	17075.602581	5.228556e+08	22866.034872	0.893060	0.839737
CatBoost	16747.423144	5.286694e+08	22992.812640	0.891871	0.839737
Gradient Boosting	17183.843300	5.527340e+08	23510.296331	0.886949	0.842758
Random Forest	18481.755980	6.442681e+08	25382.437692	0.868228	0.830443
Linear Regression	20095.463723	7.248306e+08	26922.677450	0.851750	0.826077
Decision Tree	24551.582002	1.083148e+09	32911.212588	0.778463	0.649256

Baseline vs Optimized Model Comparison

The performance of the baseline models was compared with the optimized models.

The baseline models were trained using the preprocessed dataset without feature engineering. During the optimization phase, feature engineering, feature selection, multicollinearity analysis, cross validation, and hyperparameter tuning were applied.

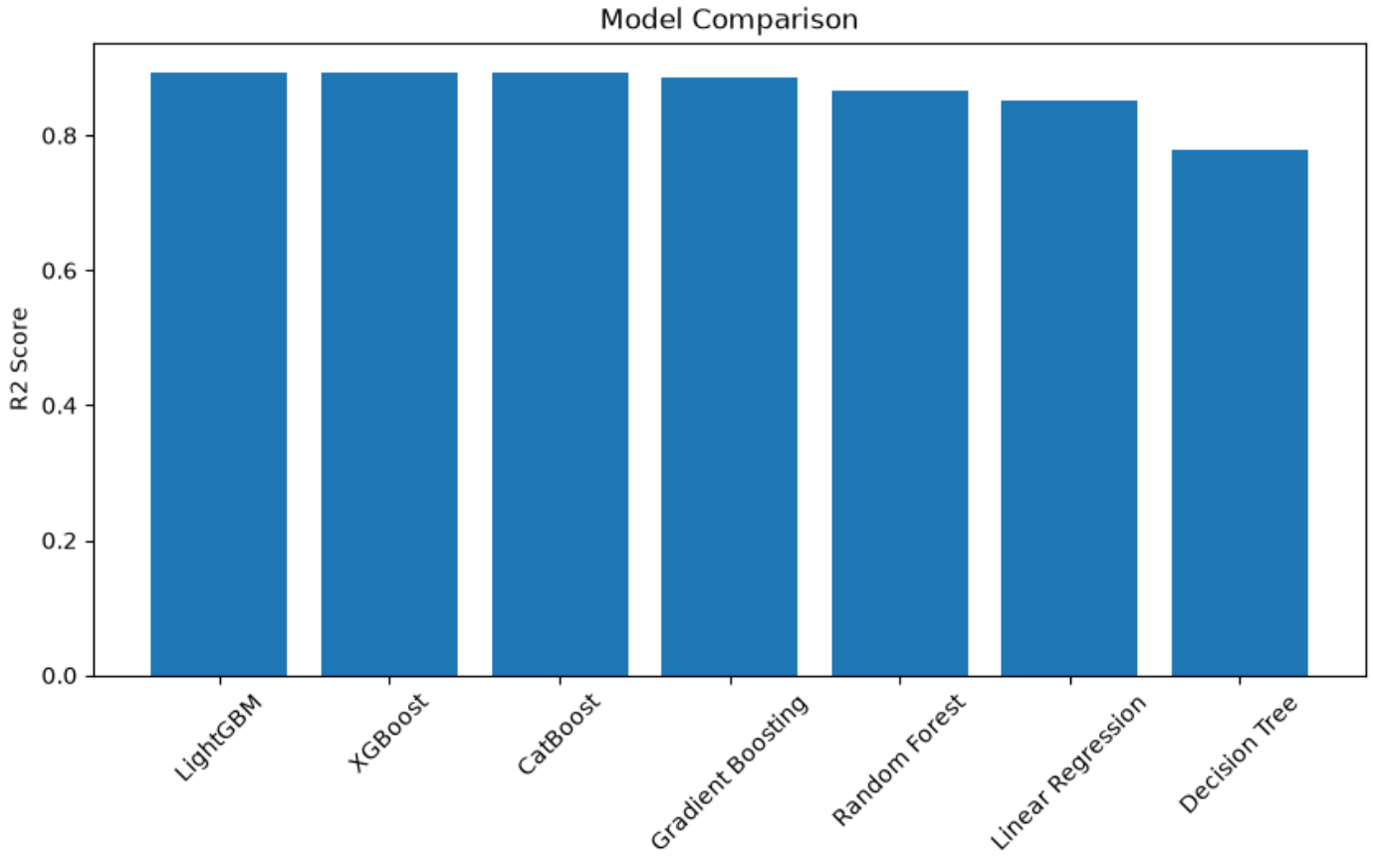
Although the optimized models produced slightly lower R^2 scores than the baseline models, they were trained using a refined feature set. This resulted in simpler models with improved generalization and reduced complexity, making them more suitable for deployment.

Model Comparison

All regression models were compared based on their evaluation metrics.

The comparison showed that ensemble learning algorithms consistently outperformed traditional regression models. Linear Regression served as a good baseline, while Decision Tree produced the lowest prediction accuracy. Random Forest, Gradient Boosting, LightGBM, CatBoost, and XGBoost demonstrated better prediction performance due to their ensemble learning approach.

During the baseline evaluation, **CatBoost Regressor** achieved the highest R^2 score. However, after applying optimization techniques, **XGBoost Regressor** produced the best overall performance and was selected as the final model.



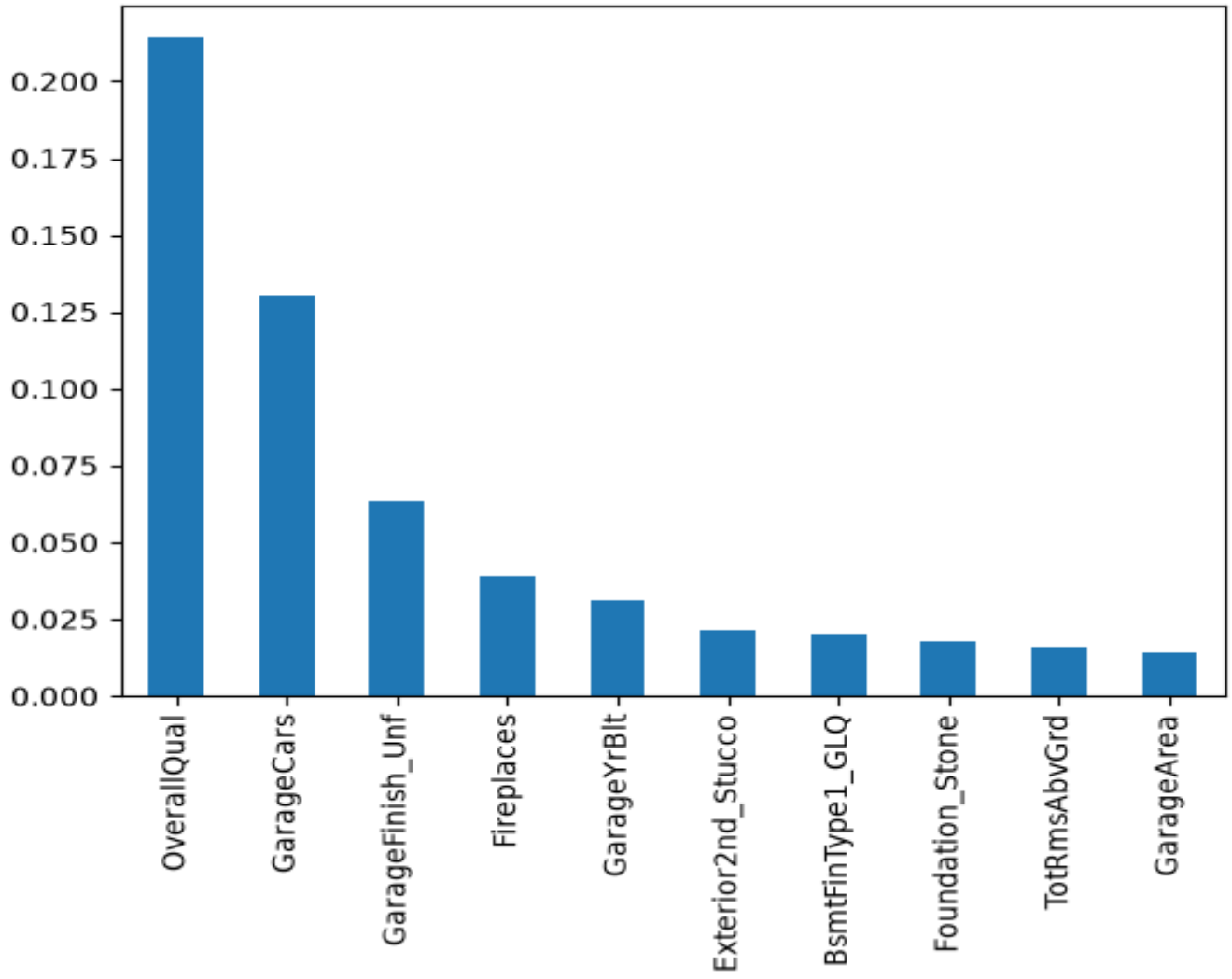
Feature Importance

Feature importance analysis was performed using the trained XGBoost model to identify the variables that contributed the most to house price prediction.

The analysis showed that the following features had the greatest influence:

- Overall Quality
- Garage Capacity
- Garage Area
- Lot Area
- Masonry Veneer Area
- Garage Built Year
- Number of Fireplaces
- Total Rooms Above Ground
- Kitchen Quality
- Basement Quality

These top 10 features were selected for the final Flask application to simplify user input while maintaining good prediction accuracy.



Model Interpretation

The evaluation results indicate that the optimized XGBoost model effectively captures the relationship between property characteristics and house prices.

Although the optimized model achieved a slightly lower R^2 score than the baseline CatBoost model, it demonstrated better stability, improved generalization, and reduced model complexity. The optimized feature set also reduced the number of inputs required from the user, making the prediction system more practical for real-world applications.

The combination of feature engineering, feature selection, and hyperparameter tuning contributed to building a reliable and efficient prediction model.

Final Model Selection

The final model was selected by considering multiple factors rather than prediction accuracy alone.

While **CatBoost Regressor** achieved the highest R^2 score during the baseline evaluation, **XGBoost Regressor** provided the best balance between prediction accuracy, stability, and model simplicity after optimization.

The reasons for selecting XGBoost include:

- Good prediction accuracy
- Lower prediction error among optimized models
- Stable Cross Validation performance
- Better generalization on unseen data
- Efficient training and prediction
- Reduced feature complexity
- Easy integration into the Flask web application

Based on these observations, XGBoost Regressor was selected as the final deployed model.

FLASK WEB APPLICATION

Introduction

After selecting the final machine learning model, the next step was to deploy it through a web application. A Flask-based application was developed to provide an interactive interface where users can enter property details and obtain an estimated house price instantly.

The application was designed using the Flask framework for the backend and Bootstrap for the frontend, resulting in a responsive and user-friendly interface.

The deployed application demonstrates the complete workflow of the project, from data preprocessing and model training to real-time prediction.

Objectives of the Web Application

The primary objectives of the web application are:

- Provide an easy-to-use interface for house price prediction.
- Allow users to enter important property details.
- Generate real-time house price predictions using the trained XGBoost model.
- Display project information and model comparison results.
- Present analytical visualizations and downloadable reports.

Technologies Used

Technology	Purpose
Python	Programming Language
Flask	Backend Web Framework
Bootstrap 5	Responsive User Interface
HTML5	Web Page Structure
CSS3	Styling
Pandas	Data Processing
XGBoost	Machine Learning Model
Joblib	Model Serialization

Application Architecture

The Flask application follows a simple three-layer architecture.

Presentation Layer

The presentation layer consists of HTML pages developed using Bootstrap. It provides a responsive interface for users to interact with the prediction system.

Application Layer

The application layer is implemented using Flask. It receives user input, validates the data, loads the trained machine learning model, performs prediction, and returns the predicted house price.

Model Layer

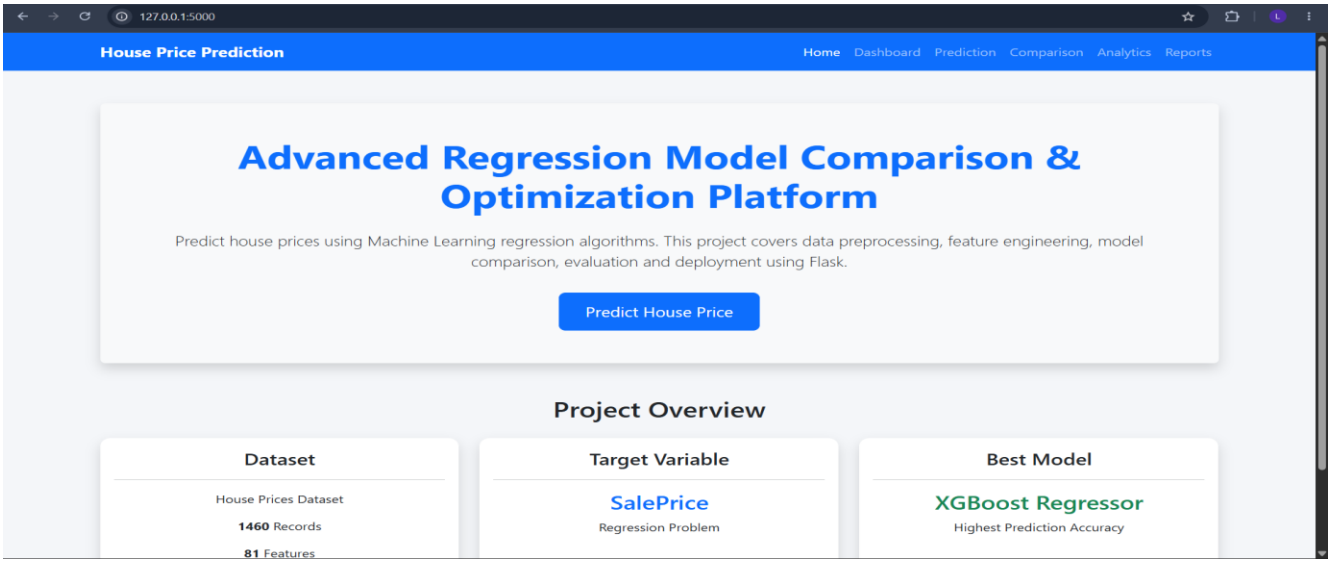
The model layer contains the trained XGBoost regression model. The model processes the user inputs and predicts the selling price based on the learned patterns from the training dataset.

Home Page

The Home page serves as the landing page of the application. It provides a brief overview of the project along with navigation links to different modules.

The page displays:

- Project Title
- Project Overview
- Dataset Information
- Target Variable
- Navigation Menu
- Quick Access to Prediction Module



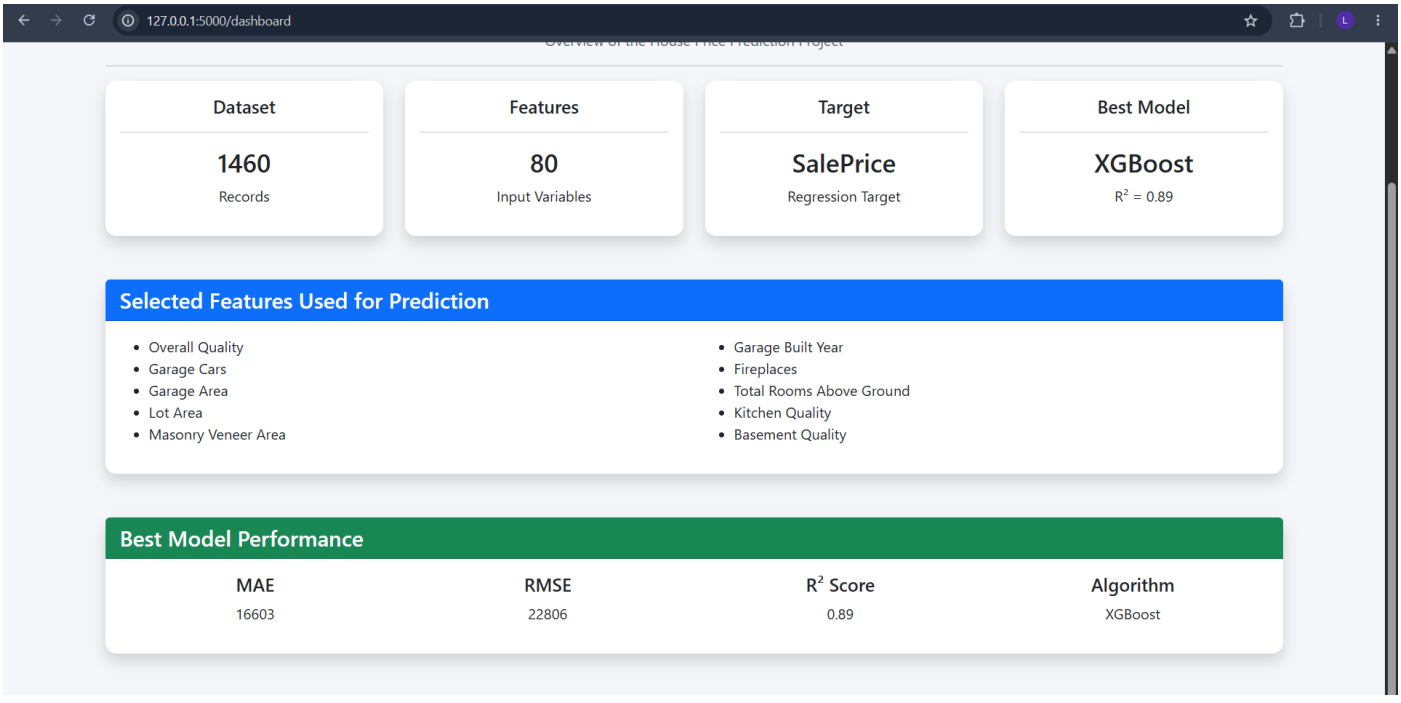
Dashboard

The Dashboard provides a summary of the machine learning project.

It displays:

- Dataset Information
- Number of Records
- Number of Features
- Target Variable
- Best Performing Model
- Overall Project Summary

This page allows users to quickly understand the project without viewing the complete report.



Dashboard Page

Prediction Module

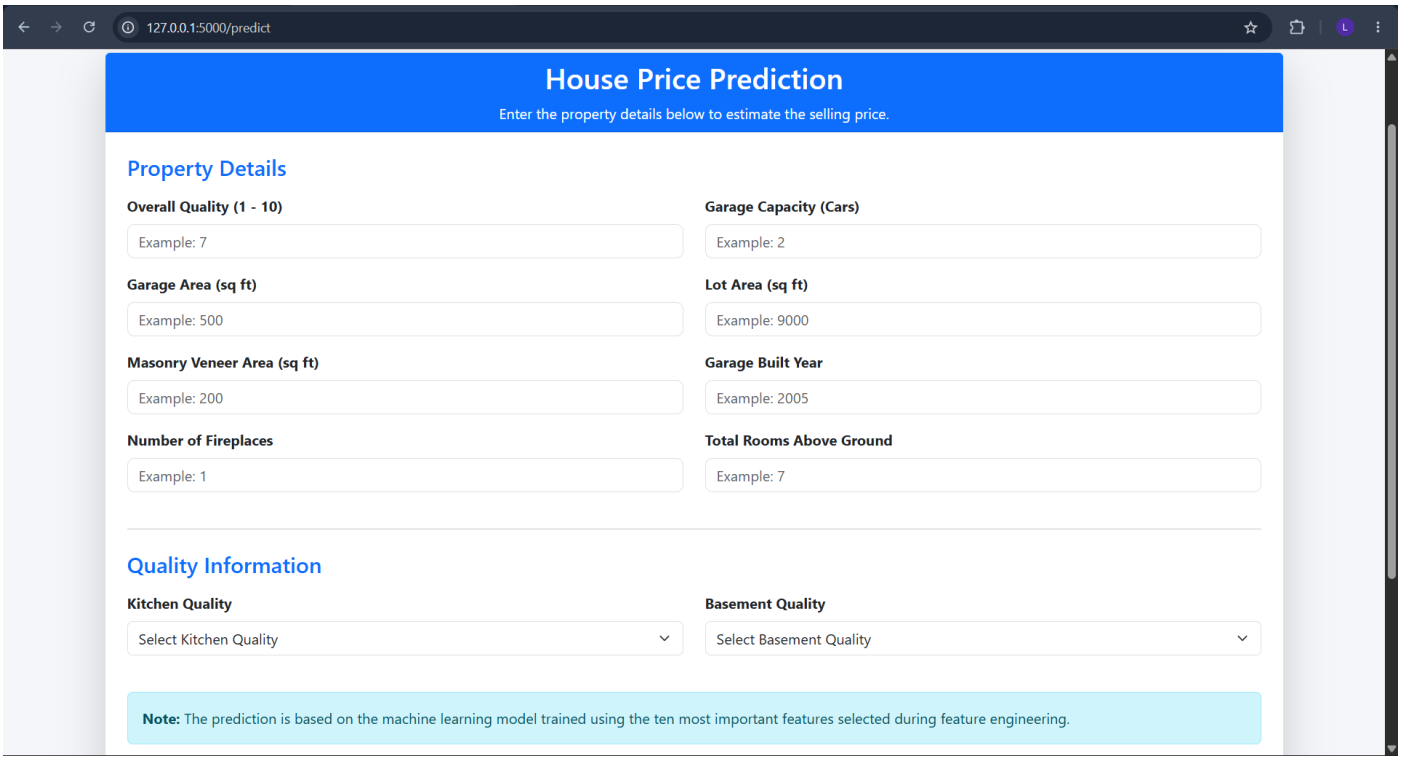
The Prediction module is the main functionality of the application.

Users enter the important property details through an input form, and the trained XGBoost model predicts the estimated selling price.

The prediction system uses the following features:

- Overall Quality
- Garage Capacity
- Garage Area
- Lot Area
- Masonry Veneer Area
- Garage Built Year
- Number of Fireplaces
- Total Rooms Above Ground
- Kitchen Quality
- Basement Quality

These features were selected based on feature importance analysis.



Prediction Page

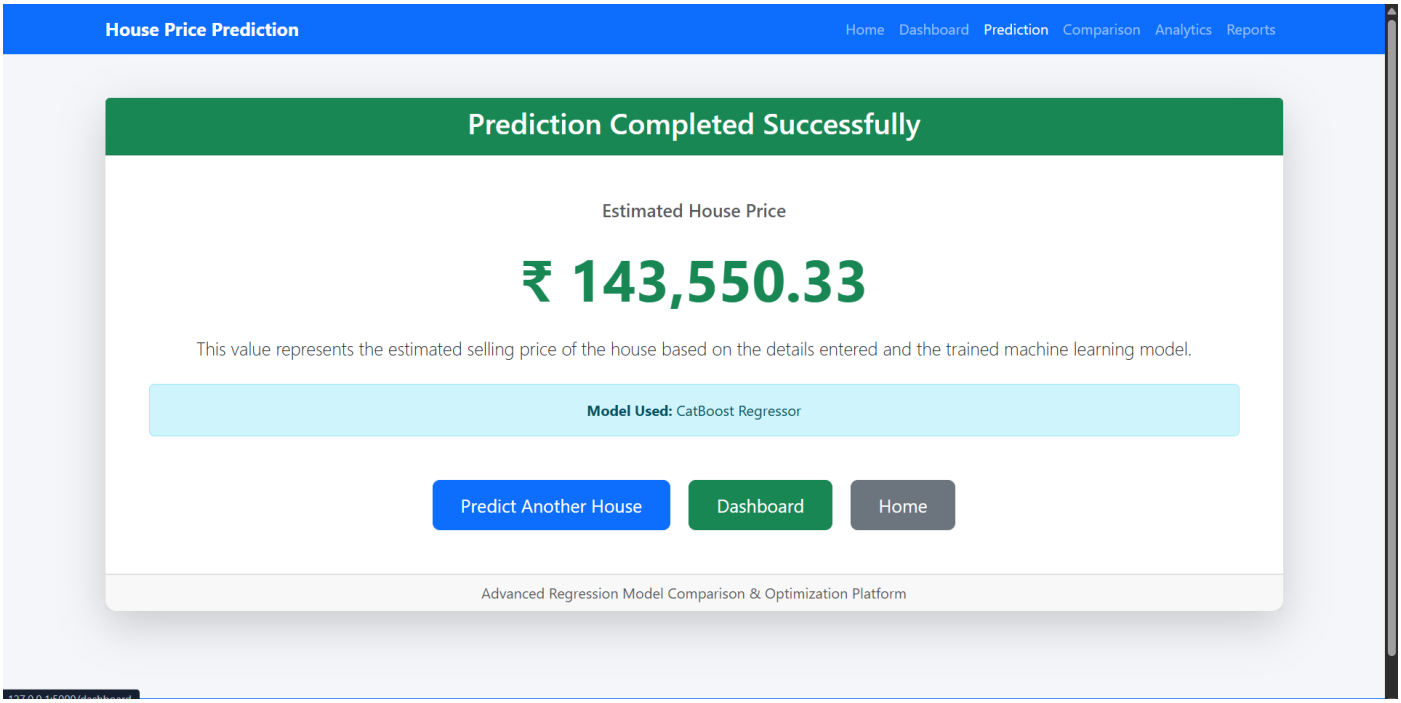
Prediction Result

After submitting the property details, the trained model processes the input values and estimates the house price.

The result page displays:

- Estimated House Price
- Predict Another House Button

This provides users with immediate feedback in a simple and readable format.



Prediction Result Page

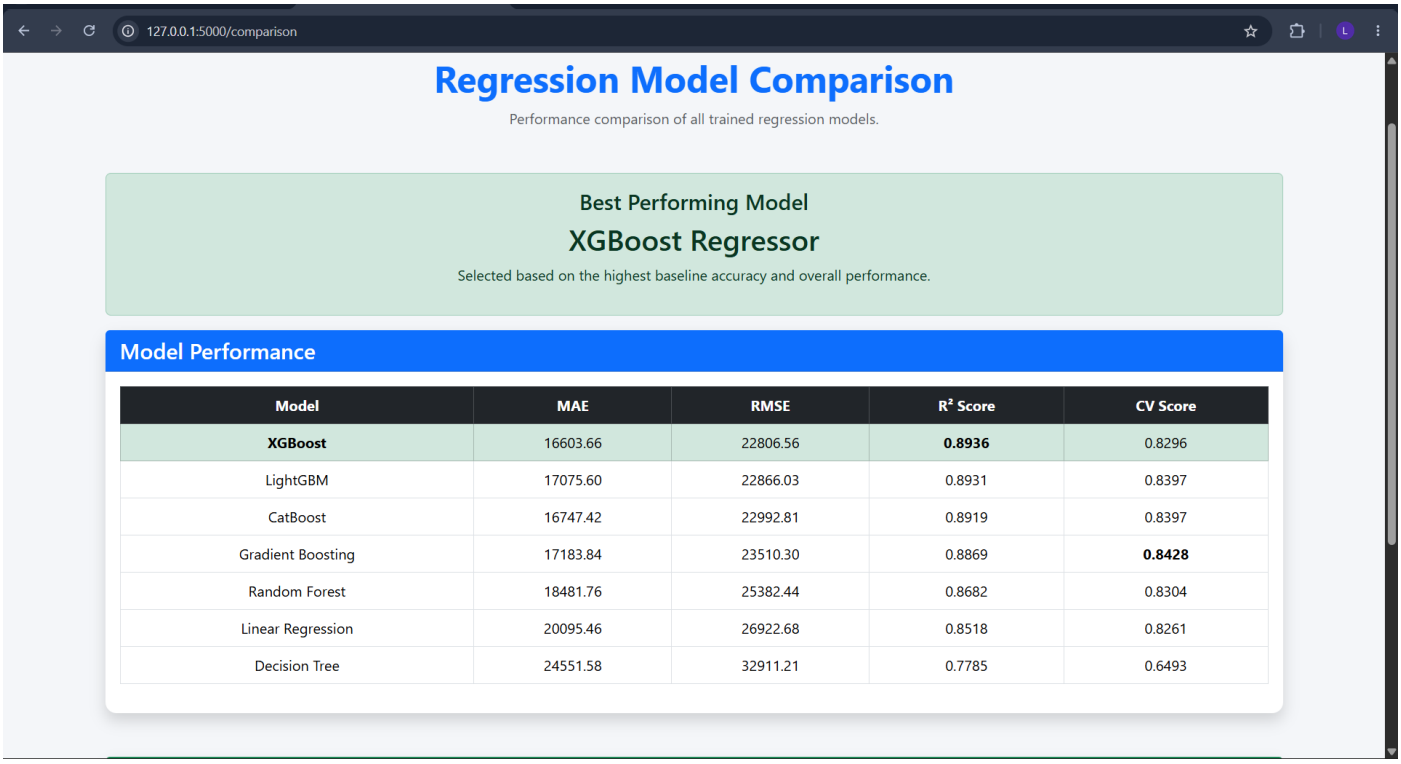
Model Comparison Page

The Model Comparison page presents the performance of all regression models developed during the project.

The page displays:

- Model Comparison Table
- MAE
- RMSE
- R^2 Score
- Cross Validation Score

The best-performing model is highlighted for easy identification.



Model Comparison Page

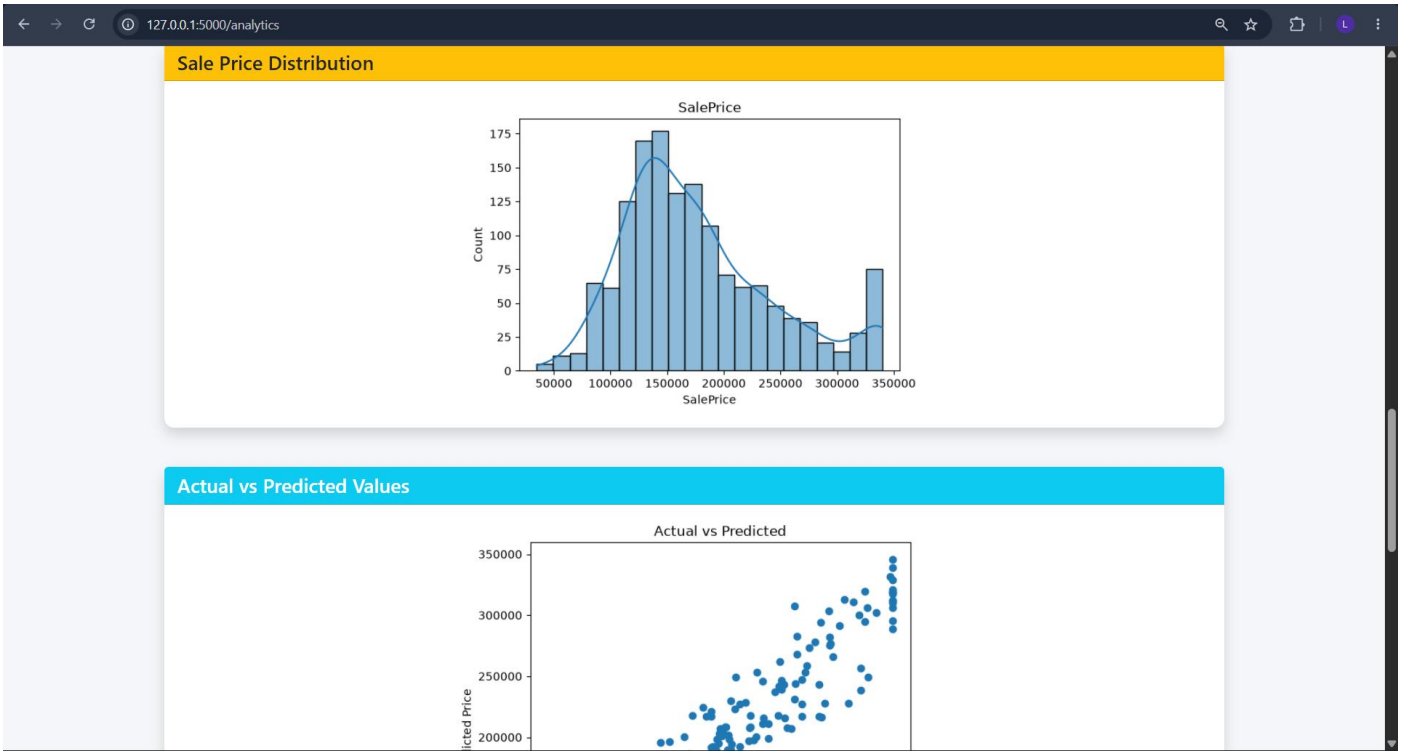
Analytics Dashboard

The Analytics page provides visual insights into the dataset and model.

The following visualizations are included:

- Correlation Heatmap
- Feature Importance Graph
- Feature Distribution
- Residual Analysis
- Actual vs Predicted Plot

These visualizations help users understand how different features influence house price prediction.



Analytics Page

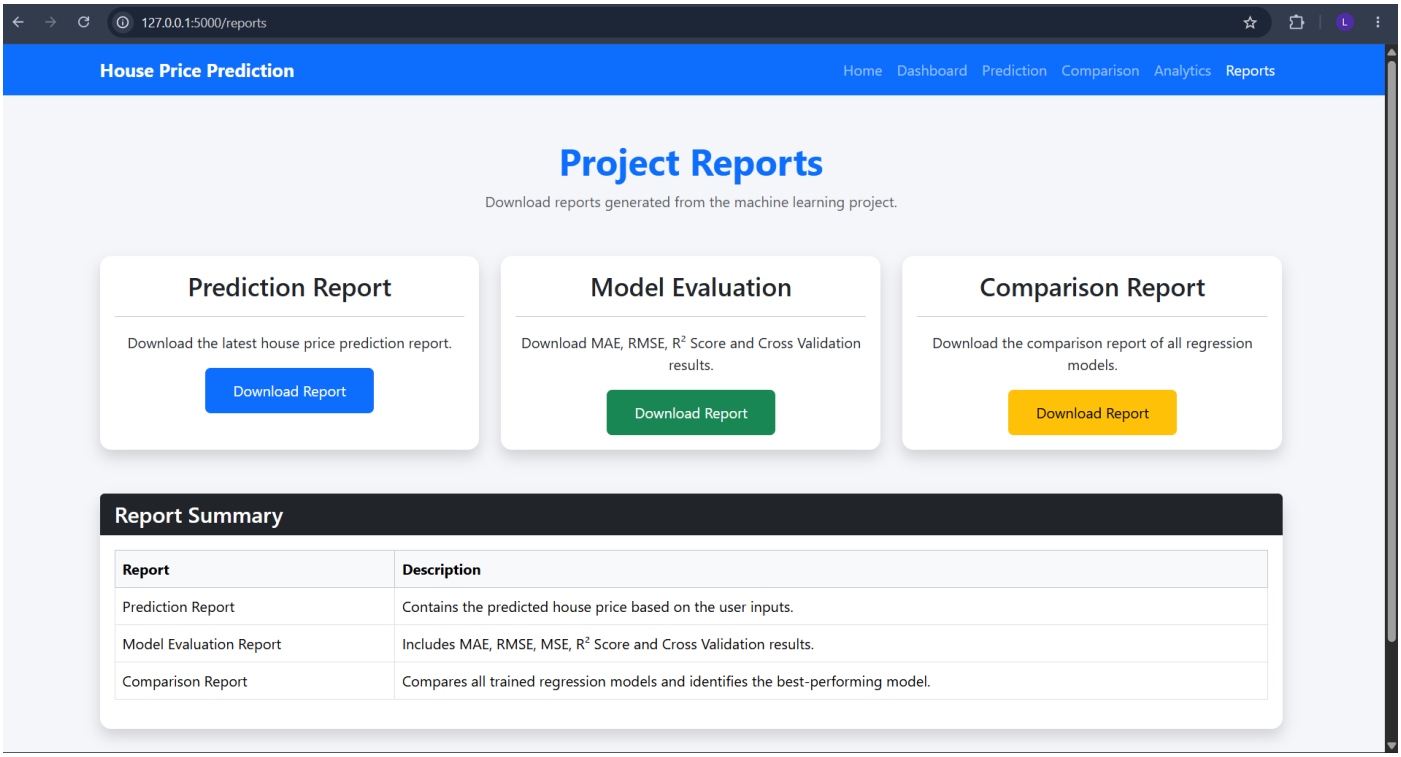
Reports Module

The Reports page provides access to project documentation.

Users can view or download:

- Model Evaluation Report
- Model Comparison Report
- Prediction Report

These reports summarize the complete machine learning workflow and evaluation results.



Report Page

Application Workflow

The overall workflow of the web application is illustrated below.

1. User opens the Flask application.
2. User navigates to the Prediction page.
3. Property details are entered through the input form.
4. Flask receives the input values.
5. Input data is converted into the required format.
6. The trained XGBoost model is loaded.
7. The model predicts the house price.
8. The predicted value is displayed to the user.
9. Users can explore analytics, model comparison, and reports.

Advantages of the Application

The developed Flask application provides several advantages:

- Simple and user-friendly interface.
- Fast prediction response.
- Uses the optimized XGBoost model.
- Requires only the top 10 important input features.

- Responsive design using Bootstrap.
- Easy navigation between project modules.
- Provides analytical visualizations and project reports.
- Can be extended for future deployment on cloud platforms.

Conclusion

This project successfully developed and compared multiple regression models for predicting house prices using the House Prices dataset. Various preprocessing and optimization techniques, including feature engineering, feature selection, cross validation, and hyperparameter tuning, were applied to improve model performance. Based on the experimental results, **XGBoost Regressor** was selected as the final model due to its strong predictive performance, better generalization, and suitability for deployment. Finally, the trained model was integrated into a Flask web application, providing a simple and interactive interface for real-time house price prediction.